VoiceCart AI — Source Code Documentation

Complete Technical Assessment Implementation Codebase

1. index.html

Description: HTML5 Layout, Modals & Voice Control HUD Interface | File Path: index.html

```
<!DOCTYPE html>
<html lang="en" class="light">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>VoiceCart AI - Voice Command Shopping Assistant</title>
    <!-- Tailwind CSS CDN -->
    <script src="https://cdn.tailwindcss.com"></script>
    <script>
        tailwind.config = {
            darkMode: 'class',
            theme: {
                extend: {
                    colors: {
                        brand: {
                            50: '#eef2ff',
                            100: '#e0e7ff',
                            500: '#6366f1',
                            600: '#4f46e5',
                            700: '#4338ca',
                        }
                    }
                }
            }
        }
    </script>
    <!-- Lucide Icons -->
    <script src="https://unpkg.com/lucide@latest"></script>
    <link rel="stylesheet" href="styles.css">
</head>
<body class="bg-slate-50 dark:bg-slate-900 text-slate-800 dark:text-slate-100 min-h-screen font
-sans antialiased transition-colors duration-300">

    <!-- Top Navigation Header -->
    <header class="sticky top-0 z-40 bg-white/85 dark:bg-slate-900/85 backdrop-blur-md border-b
 border-slate-200/80 dark:border-slate-800/80">
        <div class="max-w-6xl mx-auto px-4 sm:px-6 lg:px-8 h-16 flex items-center justify-betwe
en">
            <div class="flex items-center space-x-3">
                <div class="w-10 h-10 rounded-xl bg-gradient-to-tr from-indigo-600 via-purple-6
00 to-pink-600 flex items-center justify-center text-white shadow-lg shadow-indigo-500/25">
                    <i data-lucide="mic" class="w-5 h-5"></i>
                </div>
                <div>
                    <div class="flex items-center space-x-2">
                        <h1 class="text-lg font-bold bg-gradient-to-r from-indigo-600 via-purpl
e-600 to-pink-600 bg-clip-text text-transparent">
                            VoiceCart AI
                        </h1>
                        <span class="text-[10px] uppercase tracking-wider font-extrabold px-1.5
 py-0.5 rounded bg-indigo-100 dark:bg-indigo-950 text-indigo-700 dark:text-indigo-300">PRO</spa
n>
                    </div>
                    <p class="text-xs text-slate-500 dark:text-slate-400">Voice-Powered Shoppin
g Assistant</p>
                </div>
            </div>

            <!-- Header Action Controls -->
            <div class="flex items-center space-x-2 sm:space-x-3">
                <!-- Multilingual Language Selector -->
                <div class="relative">
                    <select id="languageSelect" class="bg-slate-100 dark:bg-slate-800 text-slat
```

```
e-700 dark:text-slate-200 text-xs sm:text-sm rounded-xl px-3 py-1.5 border border-slate-200 dar
k:border-slate-700 focus:outline-none focus:ring-2 focus:ring-indigo-500 cursor-pointer">
                        <option value="en-US">■■ English</option>
                        <option value="es-ES">■■ Español</option>
                        <option value="fr-FR">■■ Français</option>
                        <option value="de-DE">■■ Deutsch</option>
                        <option value="hi-IN">■■ ■■■■■■</option>
                        <option value="zh-CN">■■ ■■</option>
                    </select>
                </div>

                <!-- Export List Button -->
                <div class="relative group">
                    <button id="exportDropdownBtn" title="Export Shopping List" class="p-2 roun
ded-xl bg-slate-100 dark:bg-slate-800 text-slate-600 dark:text-slate-300 hover:bg-slate-200 dar
k:hover:bg-slate-700">
                        <i data-lucide="download" class="w-5 h-5"></i>
                    </button>
                    <div id="exportMenu" class="hidden absolute right-0 mt-2 w-48 bg-white dark
:bg-slate-800 rounded-xl shadow-xl border border-slate-200 dark:border-slate-700 py-1 z-50 text
-xs">
                        <button id="copyListBtn" class="w-full text-left px-4 py-2 hover:bg-sla
te-100 dark:hover:bg-slate-700 flex items-center space-x-2">
                            <i data-lucide="copy" class="w-4 h-4 text-indigo-500"></i>
                            <span>Copy as Text</span>
                        </button>
                        <button id="downloadCsvBtn" class="w-full text-left px-4 py-2 hover:bg-
slate-100 dark:hover:bg-slate-700 flex items-center space-x-2">
                            <i data-lucide="file-spread-sheet" class="w-4 h-4 text-emerald-500"
></i>
                            <span>Export CSV</span>
                        </button>
                    </div>
                </div>

                <!-- Voice-Only Mode Toggle Button -->
                <button id="toggleVoiceHudBtn" title="Hands-Free Voice Mode" class="p-2 rounded
-xl bg-indigo-50 dark:bg-indigo-950/60 text-indigo-600 dark:text-indigo-400 hover:bg-indigo-100
 dark:hover:bg-indigo-900/60">
                    <i data-lucide="smartphone" class="w-5 h-5"></i>
                </button>

                <!-- Dark Theme Toggle -->
                <button id="themeToggleBtn" title="Toggle Dark/Light Theme" class="p-2 rounded-
xl bg-slate-100 dark:bg-slate-800 text-slate-600 dark:text-slate-300 hover:bg-slate-200 dark:ho
ver:bg-slate-700">
                    <i data-lucide="moon" id="themeIcon" class="w-5 h-5"></i>
                </button>

                <!-- Settings Button -->
                <button id="openSettingsBtn" title="Assistant Settings" class="p-2 rounded-xl b
g-slate-100 dark:bg-slate-800 text-slate-600 dark:text-slate-300 hover:bg-slate-200 dark:hover:
bg-slate-700">
                    <i data-lucide="settings" class="w-5 h-5"></i>
                </button>
            </div>
        </div>
    </header>

    <!-- Main Container -->
    <main class="max-w-6xl mx-auto px-4 sm:px-6 lg:px-8 py-6 space-y-6">

        <!-- Voice Command Central Controller -->
        <section class="glass-card rounded-3xl p-6 sm:p-8 shadow-xl relative overflow-hidden te
xt-center border border-indigo-100 dark:border-indigo-900/30">
            <div class="absolute -top-20 -right-20 w-72 h-72 bg-indigo-500/10 rounded-full blur
-3xl pointer-events-none"></div>
            <div class="absolute -bottom-20 -left-20 w-72 h-72 bg-purple-500/10 rounded-full bl
ur-3xl pointer-events-none"></div>

            <div class="relative z-10 space-y-5">
                <!-- Status Badge -->
                <div class="inline-flex items-center space-x-2 px-3.5 py-1 rounded-full bg-indi
go-50 dark:bg-indigo-950/60 border border-indigo-200/80 dark:border-indigo-800/80 text-indigo-7
00 dark:text-indigo-300 text-xs font-semibold shadow-xs">
```

```
                    <span id="voiceStatusDot" class="w-2.5 h-2.5 rounded-full bg-indigo-500 ani
mate-pulse"></span>
                    <span id="voiceStatusText">Tap Mic or Say Command</span>
                </div>

                <!-- Main Interactive Voice Button -->
                <div class="flex justify-center items-center my-4">
                    <button id="micBtn" class="w-20 h-20 sm:w-24 sm:h-24 rounded-full bg-gradie
nt-to-tr from-indigo-600 via-purple-600 to-pink-600 text-white flex flex-col items-center justi
fy-center shadow-xl shadow-indigo-500/30 hover:scale-105 active:scale-95 transition-all duratio
n-300 focus:outline-none">
                        <i data-lucide="mic" id="micIcon" class="w-8 h-8 sm:w-10 sm:h-10"></i>
                        <span id="micSubtext" class="text-[10px] uppercase font-extrabold track
ing-widest mt-1">TAP TO SPEAK</span>
                    </button>
                </div>

                <!-- Audio Waveform Visualizer -->
                <div id="audioWaveform" class="hidden items-center justify-center h-8">
                    <div class="audio-bar"></div>
                    <div class="audio-bar"></div>
                    <div class="audio-bar"></div>
                    <div class="audio-bar"></div>
                    <div class="audio-bar"></div>
                </div>

                <!-- Speech Transcript Bubble -->
                <div id="transcriptBox" class="max-w-xl mx-auto min-h-[48px] p-3.5 rounded-2xl
bg-slate-100/90 dark:bg-slate-800/90 border border-slate-200 dark:border-slate-700 flex items-c
enter justify-center text-sm italic text-slate-700 dark:text-slate-200 shadow-inner">
                    <span id="transcriptText">Try saying: "Add 2 bottles of organic milk for $4
" or "Add pancake ingredients"</span>
                </div>

                <!-- Quick Recipe Presets & Voice Hints -->
                <div class="space-y-2">
                    <div class="flex flex-wrap items-center justify-center gap-2 text-xs">
                        <span class="text-slate-400 font-medium">Quick Commands:</span>
                        <button class="hint-chip bg-white dark:bg-slate-800 border border-slate
-200 dark:border-slate-700 rounded-full px-3 py-1 hover:border-indigo-400 hover:text-indigo-600
 transition-colors shadow-xs">"Add 6 bananas"</button>
                        <button class="hint-chip bg-white dark:bg-slate-800 border border-slate
-200 dark:border-slate-700 rounded-full px-3 py-1 hover:border-indigo-400 hover:text-indigo-600
 transition-colors shadow-xs">"Remove bread"</button>
                        <button class="hint-chip bg-white dark:bg-slate-800 border border-slate
-200 dark:border-slate-700 rounded-full px-3 py-1 hover:border-indigo-400 hover:text-indigo-600
 transition-colors shadow-xs">"Find items under $5"</button>
                        <button class="hint-chip bg-white dark:bg-slate-800 border border-slate
-200 dark:border-slate-700 rounded-full px-3 py-1 hover:border-indigo-400 hover:text-indigo-600
 transition-colors shadow-xs">"What should I buy?"</button>
                    </div>

                    <!-- Recipe Bundle Voice Shortcuts -->
                    <div class="flex items-center justify-center space-x-2 text-xs pt-1">
                        <span class="text-slate-400 font-medium flex items-center space-x-1">
                            <i data-lucide="utensils" class="w-3.5 h-3.5 text-amber-500"></i>
                            <span>Recipe Bundles:</span>
                        </span>
                        <button data-recipe="pancake" class="recipe-bundle-btn px-2.5 py-1 roun
ded-lg bg-amber-50 dark:bg-amber-950/50 text-amber-800 dark:text-amber-300 border border-amber-
200 dark:border-amber-800 hover:bg-amber-100 font-medium">■ Pancake Mix</button>
                        <button data-recipe="guacamole" class="recipe-bundle-btn px-2.5 py-1 ro
unded-lg bg-emerald-50 dark:bg-emerald-950/50 text-emerald-800 dark:text-emerald-300 border bor
der-emerald-200 dark:border-emerald-800 hover:bg-emerald-100 font-medium">■ Guacamole Kit</butt
on>
                        <button data-recipe="salad" class="recipe-bundle-btn px-2.5 py-1 rounde
d-lg bg-purple-50 dark:bg-purple-950/50 text-purple-800 dark:text-purple-300 border border-purp
le-200 dark:border-purple-800 hover:bg-purple-100 font-medium">■ Fresh Salad</button>
                    </div>
                </div>
            </div>
        </section>

        <!-- Budget & List Progress Dashboard Bar -->
        <section class="glass-card rounded-2xl p-4 shadow-sm grid grid-cols-1 sm:grid-cols-3 ga
```

```
p-4 text-xs">
            <div class="flex items-center space-x-3 p-3 bg-slate-100/60 dark:bg-slate-800/60 ro
unded-xl">
                <div class="w-9 h-9 rounded-lg bg-indigo-500/10 text-indigo-600 dark:text-indig
o-400 flex items-center justify-center">
                    <i data-lucide="shopping-cart" class="w-5 h-5"></i>
                </div>
                <div>
                    <p class="text-slate-400">Total Items</p>
                    <p id="dashTotalItems" class="text-sm font-bold">0 Items</p>
                </div>
            </div>

            <div class="flex items-center space-x-3 p-3 bg-slate-100/60 dark:bg-slate-800/60 ro
unded-xl">
                <div class="w-9 h-9 rounded-lg bg-emerald-500/10 text-emerald-600 dark:text-eme
rald-400 flex items-center justify-center">
                    <i data-lucide="check-circle-2" class="w-5 h-5"></i>
                </div>
                <div class="w-full">
                    <div class="flex justify-between items-center mb-1">
                        <span class="text-slate-400">Completed</span>
                        <span id="dashCompletedPercent" class="font-bold text-emerald-600">0%</
span>
                    </div>
                    <div class="w-full bg-slate-200 dark:bg-slate-700 h-1.5 rounded-full overfl
ow-hidden">
                        <div id="dashProgressBar" class="bg-emerald-500 h-full w-0 transition-a
ll duration-500"></div>
                    </div>
                </div>
            </div>

            <div class="flex items-center space-x-3 p-3 bg-slate-100/60 dark:bg-slate-800/60 ro
unded-xl">
                <div class="w-9 h-9 rounded-lg bg-amber-500/10 text-amber-600 dark:text-amber-4
00 flex items-center justify-center">
                    <i data-lucide="dollar-sign" class="w-5 h-5"></i>
                </div>
                <div>
                    <p class="text-slate-400">Estimated Total</p>
                    <p id="dashEstTotal" class="text-sm font-bold text-slate-900 dark:text-slat
e-100">$0.00</p>
                </div>
            </div>
        </section>

        <!-- Smart Suggestions Accordion -->
        <section class="glass-card rounded-2xl p-5 shadow-sm space-y-4">
            <div class="flex items-center justify-between">
                <div class="flex items-center space-x-2">
                    <i data-lucide="sparkles" class="w-5 h-5 text-amber-500"></i>
                    <h2 class="text-base font-semibold">Smart Suggestions</h2>
                    <span id="seasonBadge" class="text-xs px-2.5 py-0.5 rounded-full bg-amber-1
00 dark:bg-amber-950/60 text-amber-800 dark:text-amber-300 font-medium">Summer</span>
                </div>
                <button id="refreshSuggestionsBtn" class="text-xs text-indigo-600 dark:text-ind
igo-400 hover:underline flex items-center space-x-1">
                    <i data-lucide="refresh-cw" class="w-3.5 h-3.5"></i>
                    <span>Refresh</span>
                </button>
            </div>

            <!-- Suggestions Grid -->
            <div id="suggestionsContainer" class="grid grid-cols-1 md:grid-cols-3 gap-3">
                <!-- Dynamically populated suggestions -->
            </div>
        </section>

        <!-- Shopping List Controls: Search & Category Tabs -->
        <section class="glass-card rounded-2xl p-5 shadow-sm space-y-4">
            <div class="flex flex-col sm:flex-row items-center justify-between gap-3">
                <div class="relative w-full sm:w-80">
                    <i data-lucide="search" class="w-4 h-4 absolute left-3.5 top-1/2 -translate
-y-1/2 text-slate-400"></i>
                    <input type="text" id="searchInput" placeholder="Search items, brands, or '
```

```
under $5'..." class="w-full pl-9 pr-8 py-2 text-sm bg-slate-100 dark:bg-slate-800 border border
-slate-200 dark:border-slate-700 rounded-xl focus:outline-none focus:ring-2 focus:ring-indigo-5
00">
                    <button id="clearSearchBtn" class="hidden absolute right-2.5 top-1/2 -trans
late-y-1/2 text-slate-400 hover:text-slate-600">
                        <i data-lucide="x" class="w-4 h-4"></i>
                    </button>
                </div>

                <div class="flex items-center space-x-2 w-full sm:w-auto">
                    <button id="openAddItemModalBtn" class="w-full sm:w-auto px-4 py-2 bg-indig
o-600 hover:bg-indigo-700 text-white rounded-xl text-sm font-medium flex items-center justify-c
enter space-x-2 shadow-sm">
                        <i data-lucide="plus" class="w-4 h-4"></i>
                        <span>Add Item</span>
                    </button>
                </div>
            </div>

            <!-- Active Filter Banner -->
            <div id="activeFilterBanner" class="hidden items-center justify-between px-3.5 py-2
 bg-indigo-50 dark:bg-indigo-950/40 rounded-xl text-xs text-indigo-700 dark:text-indigo-300 bor
der border-indigo-200 dark:border-indigo-800">
                <span id="activeFilterText">Active filter: "organic under $5"</span>
                <button id="resetFilterBtn" class="font-bold underline hover:text-indigo-900">R
eset</button>
            </div>

            <!-- Category Filter Horizontal Tabs -->
            <div class="flex items-center space-x-2 overflow-x-auto pb-1 scrollbar-none text-xs
" id="categoryFilterTabs">
                <button data-category="ALL" class="cat-tab bg-indigo-600 text-white px-3.5 py-1
.5 rounded-lg font-medium whitespace-nowrap">All Items</button>
                <button data-category="Produce" class="cat-tab bg-slate-100 dark:bg-slate-800 t
ext-slate-600 dark:text-slate-300 px-3.5 py-1.5 rounded-lg font-medium hover:bg-slate-200 white
space-nowrap">■ Produce</button>
                <button data-category="Dairy" class="cat-tab bg-slate-100 dark:bg-slate-800 tex
t-slate-600 dark:text-slate-300 px-3.5 py-1.5 rounded-lg font-medium hover:bg-slate-200 whitesp
ace-nowrap">■ Dairy</button>
                <button data-category="Bakery" class="cat-tab bg-slate-100 dark:bg-slate-800 te
xt-slate-600 dark:text-slate-300 px-3.5 py-1.5 rounded-lg font-medium hover:bg-slate-200 whites
pace-nowrap">■ Bakery</button>
                <button data-category="Meat & Seafood" class="cat-tab bg-slate-100 dark:bg-slat
e-800 text-slate-600 dark:text-slate-300 px-3.5 py-1.5 rounded-lg font-medium hover:bg-slate-20
0 whitespace-nowrap">■ Meat</button>
                <button data-category="Pantry" class="cat-tab bg-slate-100 dark:bg-slate-800 te
xt-slate-600 dark:text-slate-300 px-3.5 py-1.5 rounded-lg font-medium hover:bg-slate-200 whites
pace-nowrap">■ Pantry</button>
                <button data-category="Beverages" class="cat-tab bg-slate-100 dark:bg-slate-800
 text-slate-600 dark:text-slate-300 px-3.5 py-1.5 rounded-lg font-medium hover:bg-slate-200 whi
tespace-nowrap">■ Beverages</button>
                <button data-category="Snacks" class="cat-tab bg-slate-100 dark:bg-slate-800 te
xt-slate-600 dark:text-slate-300 px-3.5 py-1.5 rounded-lg font-medium hover:bg-slate-200 whites
pace-nowrap">■ Snacks</button>
                <button data-category="Personal Care" class="cat-tab bg-slate-100 dark:bg-slate
-800 text-slate-600 dark:text-slate-300 px-3.5 py-1.5 rounded-lg font-medium hover:bg-slate-200
 whitespace-nowrap">■ Care</button>
                <button data-category="Household" class="cat-tab bg-slate-100 dark:bg-slate-800
 text-slate-600 dark:text-slate-300 px-3.5 py-1.5 rounded-lg font-medium hover:bg-slate-200 whi
tespace-nowrap">■ Household</button>
            </div>
        </section>

        <!-- Shopping List Items Container -->
        <section class="space-y-4">
            <div class="flex items-center justify-between">
                <h2 class="text-lg font-bold flex items-center space-x-2">
                    <span>Shopping List</span>
                    <span id="itemCountBadge" class="text-xs bg-indigo-100 dark:bg-indigo-950 t
ext-indigo-700 dark:text-indigo-300 px-2.5 py-0.5 rounded-full font-bold">4 items</span>
                </h2>

                <div class="flex items-center space-x-3 text-xs">
                    <button id="clearCompletedBtn" class="text-slate-500 hover:text-slate-800 d
ark:hover:text-slate-200 font-medium">Clear Completed</button>
                    <span class="text-slate-300">|</span>
```

```
                    <button id="clearAllBtn" class="text-rose-500 hover:text-rose-700 font-medi
um">Clear All</button>
                </div>
            </div>

            <!-- Items Card List -->
            <div id="shoppingListContainer" class="space-y-3">
                <!-- Dynamically rendered list items -->
            </div>

            <!-- Empty State Placeholder -->
            <div id="emptyListState" class="hidden glass-card rounded-2xl p-12 text-center spac
e-y-3">
                <div class="w-16 h-16 mx-auto rounded-full bg-slate-100 dark:bg-slate-800 flex
items-center justify-center text-slate-400">
                    <i data-lucide="shopping-bag" class="w-8 h-8"></i>
                </div>
                <h3 class="font-bold text-base">Your shopping list is empty</h3>
                <p class="text-xs text-slate-500 max-w-sm mx-auto">Tap the microphone above and
 say <span class="font-semibold text-indigo-600">"Add milk and apples"</span> to quickly build
your list!</p>
            </div>

            <!-- Shopping List Summary Footer -->
            <div class="glass-card rounded-2xl p-4 flex flex-col sm:flex-row items-center justi
fy-between text-sm gap-2">
                <div class="flex items-center space-x-4 text-slate-600 dark:text-slate-400 text
-xs sm:text-sm">
                    <div>Total: <span id="summaryTotalCount" class="font-bold text-slate-900 da
rk:text-slate-100">0</span> items</div>
                    <div>Remaining: <span id="summaryPendingCount" class="font-bold text-indigo
-600 dark:text-indigo-400">0</span></div>
                </div>
                <div class="text-sm font-semibold text-slate-800 dark:text-slate-200">
                    Est. Total: <span id="summaryTotalPrice" class="text-base font-bold text-em
erald-600 dark:text-emerald-400">$0.00</span>
                </div>
            </div>
        </section>
    </main>

    <!-- Modal: Add Item Manual Fallback -->
    <div id="addItemModal" class="hidden fixed inset-0 z-50 bg-slate-900/60 backdrop-blur-sm fl
ex items-center justify-center p-4">
        <div class="bg-white dark:bg-slate-900 rounded-2xl max-w-md w-full p-6 shadow-2xl space
-y-4 border border-slate-200 dark:border-slate-800">
            <div class="flex items-center justify-between">
                <h3 class="text-base font-bold">Add New Shopping Item</h3>
                <button id="closeAddItemModalBtn" class="text-slate-400 hover:text-slate-600">
                    <i data-lucide="x" class="w-5 h-5"></i>
                </button>
            </div>
            <form id="manualAddForm" class="space-y-3">
                <div>
                    <label class="block text-xs font-medium mb-1">Item Name *</label>
                    <input type="text" id="manualNameInput" required placeholder="e.g. Organic
Almond Milk" class="w-full px-3 py-2 text-sm bg-slate-100 dark:bg-slate-800 border border-slate
-200 dark:border-slate-700 rounded-xl focus:ring-2 focus:ring-indigo-500 focus:outline-none">
                </div>
                <div class="grid grid-cols-2 gap-3">
                    <div>
                        <label class="block text-xs font-medium mb-1">Quantity</label>
                        <input type="number" id="manualQtyInput" min="1" value="1" class="w-ful
l px-3 py-2 text-sm bg-slate-100 dark:bg-slate-800 border border-slate-200 dark:border-slate-70
0 rounded-xl focus:ring-2 focus:ring-indigo-500 focus:outline-none">
                    </div>
                    <div>
                        <label class="block text-xs font-medium mb-1">Unit</label>
                        <select id="manualUnitSelect" class="w-full px-3 py-2 text-sm bg-slate-
100 dark:bg-slate-800 border border-slate-200 dark:border-slate-700 rounded-xl focus:ring-2 foc
us:ring-indigo-500 focus:outline-none">
                            <option value="pcs">pcs</option>
                            <option value="bottle">bottle</option>
                            <option value="gallon">gallon</option>
                            <option value="pack">pack</option>
                            <option value="box">box</option>
```

```
                            <option value="loaf">loaf</option>
                            <option value="kg">kg</option>
                            <option value="lbs">lbs</option>
                        </select>
                    </div>
                </div>
                <div class="grid grid-cols-2 gap-3">
                    <div>
                        <label class="block text-xs font-medium mb-1">Category</label>
                        <select id="manualCategorySelect" class="w-full px-3 py-2 text-sm bg-sl
ate-100 dark:bg-slate-800 border border-slate-200 dark:border-slate-700 rounded-xl focus:ring-2
 focus:ring-indigo-500 focus:outline-none">
                            <option value="Produce">Produce</option>
                            <option value="Dairy">Dairy</option>
                            <option value="Bakery">Bakery</option>
                            <option value="Meat & Seafood">Meat & Seafood</option>
                            <option value="Pantry">Pantry</option>
                            <option value="Beverages">Beverages</option>
                            <option value="Snacks">Snacks</option>
                            <option value="Personal Care">Personal Care</option>
                            <option value="Household">Household</option>
                        </select>
                    </div>
                    <div>
                        <label class="block text-xs font-medium mb-1">Price ($)</label>
                        <input type="number" id="manualPriceInput" step="0.01" placeholder="e.g
. 3.99" class="w-full px-3 py-2 text-sm bg-slate-100 dark:bg-slate-800 border border-slate-200
dark:border-slate-700 rounded-xl focus:ring-2 focus:ring-indigo-500 focus:outline-none">
                    </div>
                </div>
                <div>
                    <label class="block text-xs font-medium mb-1">Brand (Optional)</label>
                    <input type="text" id="manualBrandInput" placeholder="e.g. Silk / Horizon"
class="w-full px-3 py-2 text-sm bg-slate-100 dark:bg-slate-800 border border-slate-200 dark:bor
der-slate-700 rounded-xl focus:ring-2 focus:ring-indigo-500 focus:outline-none">
                </div>
                <div class="pt-2 flex justify-end space-x-2">
                    <button type="button" id="cancelAddItemModalBtn" class="px-4 py-2 text-xs f
ont-medium text-slate-500 hover:text-slate-800">Cancel</button>
                    <button type="submit" class="px-4 py-2 text-xs font-medium bg-indigo-600 ho
ver:bg-indigo-700 text-white rounded-xl shadow-sm">Save Item</button>
                </div>
            </form>
        </div>
    </div>

    <!-- Modal: Settings & Gemini AI Configuration -->
    <div id="settingsModal" class="hidden fixed inset-0 z-50 bg-slate-900/60 backdrop-blur-sm f
lex items-center justify-center p-4">
        <div class="bg-white dark:bg-slate-900 rounded-2xl max-w-md w-full p-6 shadow-2xl space
-y-4 border border-slate-200 dark:border-slate-800">
            <div class="flex items-center justify-between">
                <h3 class="text-base font-bold flex items-center space-x-2">
                    <i data-lucide="settings" class="w-5 h-5 text-indigo-600"></i>
                    <span>Assistant Settings</span>
                </h3>
                <button id="closeSettingsModalBtn" class="text-slate-400 hover:text-slate-600">
                    <i data-lucide="x" class="w-5 h-5"></i>
                </button>
            </div>
            <div class="space-y-4 text-xs">
                <div>
                    <label class="block font-medium mb-1">Voice Feedback (Text-to-Speech)</labe
l>
                    <div class="flex items-center justify-between p-3 bg-slate-100 dark:bg-slat
e-800 rounded-xl">
                        <span>Enable voice responses</span>
                        <input type="checkbox" id="voiceFeedbackToggle" class="w-4 h-4 text-ind
igo-600 rounded">
                    </div>
                </div>
                <div>
                    <label class="block font-medium mb-1">UI Sound Effects</label>
                    <div class="flex items-center justify-between p-3 bg-slate-100 dark:bg-slat
e-800 rounded-xl">
```

```
                        <span>Enable Web Audio UI chimes</span>
                        <input type="checkbox" id="soundFxToggle" checked class="w-4 h-4 text-i
ndigo-600 rounded">
                    </div>
                </div>
                <div>
                    <label class="block font-medium mb-1">Speech Speed Rate</label>
                    <input type="range" id="voiceSpeedRange" min="0.5" max="1.5" step="0.1" val
ue="1.0" class="w-full">
                    <div class="flex justify-between text-[10px] text-slate-400 mt-1">
                        <span>0.5x (Slow)</span>
                        <span id="speedValueText">1.0x (Normal)</span>
                        <span>1.5x (Fast)</span>
                    </div>
                </div>
                <div>
                    <label class="block font-medium mb-1">Google Gemini API Key (Optional AI En
hancement)</label>
                    <input type="password" id="geminiApiKeyInput" placeholder="AIzaSy..." class
="w-full px-3 py-2 bg-slate-100 dark:bg-slate-800 border border-slate-200 dark:border-slate-700
 rounded-xl focus:ring-2 focus:ring-indigo-500 focus:outline-none">
                    <p class="text-[10px] text-slate-400 mt-1">Leave empty to use built-in loca
l NLP parsing engine.</p>
                </div>
            </div>

            <div class="pt-2 flex justify-end">
                <button id="saveSettingsBtn" class="px-4 py-2 text-xs font-medium bg-indigo-600
 hover:bg-indigo-700 text-white rounded-xl shadow-sm">Save Preferences</button>
            </div>
        </div>
    </div>

    <!-- Voice-Only Mode HUD Overlay -->
    <div id="voiceHudOverlay" class="hidden voice-hud-overlay">
        <button id="exitVoiceHudBtn" class="absolute top-6 right-6 p-3 rounded-full bg-white/10
 hover:bg-white/20 text-white">
            <i data-lucide="x" class="w-6 h-6"></i>
        </button>

        <div class="text-center space-y-6 max-w-lg">
            <div class="inline-flex items-center space-x-2 px-4 py-1.5 rounded-full bg-indigo-5
00/20 border border-indigo-400/30 text-indigo-300 text-sm font-semibold">
                <span class="w-2.5 h-2.5 rounded-full bg-indigo-400 animate-ping"></span>
                <span>Hands-Free Voice Mode</span>
            </div>

            <div class="my-6">
                <button id="hudMicBtn" class="w-32 h-32 rounded-full bg-gradient-to-tr from-ind
igo-500 via-purple-500 to-pink-500 text-white flex items-center justify-center shadow-2xl shado
w-indigo-500/50 hover:scale-105 transition-all">
                    <i data-lucide="mic" class="w-14 h-14"></i>
                </button>
            </div>

            <div id="hudTranscriptBox" class="p-4 rounded-2xl bg-white/10 border border-white/2
0 text-lg font-medium italic min-h-[64px] flex items-center justify-center">
                Say a command, e.g., "Add 2 bottles of water"
            </div>

            <div class="text-xs text-slate-400 space-y-1">
                <p>Language: <span id="hudLangText" class="text-white font-semibold">English (U
S)</span></p>
                <p>Speak clearly into your device's microphone</p>
            </div>
        </div>
    </div>

    <!-- Toast Notification Container -->
    <div id="toastContainer" class="fixed bottom-6 right-6 z-50 space-y-2 pointer-events-none">
</div>

    <!-- JavaScript Controllers (ES Modules) -->
    <script type="module" src="js/app.js"></script>
</body>
</html>
```

2. styles.css

Description: CSS Styling, Glassmorphism, Badge Colors & Visualizer Animations | File Path: styles.css

```
/* Custom Styling for Voice Command Shopping Assistant (VoiceCart AI) */

@keyframes pulse-mic {
    0% {
        box-shadow: 0 0 0 0 rgba(79, 70, 229, 0.7), 0 0 0 0 rgba(147, 51, 234, 0.4);
        transform: scale(1);
    }
    50% {
        box-shadow: 0 0 0 20px rgba(79, 70, 229, 0), 0 0 0 35px rgba(147, 51, 234, 0);
        transform: scale(1.06);
    }
    100% {
        box-shadow: 0 0 0 0 rgba(79, 70, 229, 0), 0 0 0 0 rgba(147, 51, 234, 0);
        transform: scale(1);
    }
}

@keyframes wave-bar {
    0%, 100% { height: 8px; }
    50% { height: 32px; }
}

.mic-active {
    animation: pulse-mic 1.5s infinite ease-in-out;
    background: linear-gradient(135deg, #4f46e5 0%, #7c3aed 50%, #db2777 100%) !important;
}

.audio-bar {
    width: 4px;
    background-color: #6366f1;
    border-radius: 4px;
    display: inline-block;
    margin: 0 2px;
}

.audio-bar:nth-child(1) { animation: wave-bar 1.2s infinite ease-in-out 0.1s; }
.audio-bar:nth-child(2) { animation: wave-bar 1.2s infinite ease-in-out 0.3s; }
.audio-bar:nth-child(3) { animation: wave-bar 1.2s infinite ease-in-out 0.2s; }
.audio-bar:nth-child(4) { animation: wave-bar 1.2s infinite ease-in-out 0.4s; }
.audio-bar:nth-child(5) { animation: wave-bar 1.2s infinite ease-in-out 0.15s; }

/* Custom Category Badge Colors */
.badge-produce { background-color: #dcfce7; color: #166534; }
.badge-dairy { background-color: #e0f2fe; color: #075985; }
.badge-bakery { background-color: #fef3c7; color: #92400e; }
.badge-meat { background-color: #ffe4e6; color: #9f1239; }
.badge-pantry { background-color: #f3e8ff; color: #6b21a8; }
.badge-beverages { background-color: #ccfbf1; color: #115e59; }
.badge-snacks { background-color: #ffedd5; color: #9a3412; }
.badge-household { background-color: #f1f5f9; color: #334155; }
.badge-personal { background-color: #fce7f3; color: #9d174d; }

.dark .badge-produce { background-color: rgba(22, 101, 52, 0.3); color: #86efac; }
.dark .badge-dairy { background-color: rgba(7, 89, 133, 0.3); color: #7dd3fc; }
.dark .badge-bakery { background-color: rgba(146, 64, 14, 0.3); color: #fde047; }
.dark .badge-meat { background-color: rgba(159, 18, 57, 0.3); color: #fca5a5; }
.dark .badge-pantry { background-color: rgba(107, 33, 168, 0.3); color: #d8b4fe; }
.dark .badge-beverages { background-color: rgba(17, 94, 89, 0.3); color: #5eead4; }
.dark .badge-snacks { background-color: rgba(154, 52, 18, 0.3); color: #fdba74; }
.dark .badge-household { background-color: rgba(51, 65, 85, 0.3); color: #cbd5e1; }
.dark .badge-personal { background-color: rgba(157, 23, 77, 0.3); color: #f472b6; }

/* Completed item strikethrough styling */
.item-completed {
    opacity: 0.5;
}
.item-completed .item-title {
    text-decoration: line-through;
```

```
}

/* Glassmorphism effects */
.glass-card {
    background: rgba(255, 255, 255, 0.88);
    backdrop-filter: blur(16px);
    -webkit-backdrop-filter: blur(16px);
    border: 1px solid rgba(229, 231, 235, 0.8);
}

.dark .glass-card {
    background: rgba(15, 23, 42, 0.88);
    border: 1px solid rgba(51, 65, 85, 0.7);
}

/* Voice-Only Mode HUD Overlay */
.voice-hud-overlay {
    position: fixed;
    inset: 0;
    z-index: 9999;
    background: radial-gradient(circle at center, #1e1b4b 0%, #0f172a 100%);
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    color: white;
    padding: 2rem;
}

/* Custom Scrollbars */
.scrollbar-none::-webkit-scrollbar {
    display: none;
}
.scrollbar-none {
    -ms-overflow-style: none;
    scrollbar-width: none;
}

/* Smooth Transitions */
* {
    transition: background-color 0.2s ease, border-color 0.2s ease, color 0.2s ease, transform
0.15s ease;
}
```

3. js/app.js

Description: Main Application Controller & Event Dispatcher | File Path: js/app.js

```
/**
 * App Controller - Main application orchestration module
 * Connects VoiceManager, NLPParser, SuggestionsEngine, SoundFX, and StorageManager with DOM UI
.
 */

import { StorageManager } from './storage.js';
import { SuggestionsEngine } from './suggestions.js';
import { NLPParser, RECIPE_BUNDLES } from './nlp.js';
import { VoiceManager } from './voice.js';
import { SoundFX } from './sound.js';

class App {
    constructor() {
        this.items = StorageManager.getItems();
        this.activeCategory = 'ALL';
        this.searchQuery = '';
        this.maxPriceFilter = null;
        this.soundEnabled = true;
        this.voiceManager = null;

        this.initDOM();
        this.initVoice();
        this.initEventListeners();
        this.render();
    }

    /**
     * Cache DOM Elements
     */
    initDOM() {
        this.micBtn = document.getElementById('micBtn');
        this.micIcon = document.getElementById('micIcon');
        this.micSubtext = document.getElementById('micSubtext');
        this.voiceStatusDot = document.getElementById('voiceStatusDot');
        this.voiceStatusText = document.getElementById('voiceStatusText');
        this.audioWaveform = document.getElementById('audioWaveform');
        this.transcriptBox = document.getElementById('transcriptBox');
        this.transcriptText = document.getElementById('transcriptText');

        this.languageSelect = document.getElementById('languageSelect');
        this.themeToggleBtn = document.getElementById('themeToggleBtn');
        this.themeIcon = document.getElementById('themeIcon');
        this.toggleVoiceHudBtn = document.getElementById('toggleVoiceHudBtn');
        this.voiceHudOverlay = document.getElementById('voiceHudOverlay');

        this.exitVoiceHudBtn = document.getElementById('exitVoiceHudBtn');
        this.hudMicBtn = document.getElementById('hudMicBtn');
        this.hudTranscriptBox = document.getElementById('hudTranscriptBox');
        this.hudLangText = document.getElementById('hudLangText');

        // Export Menu
        this.exportDropdownBtn = document.getElementById('exportDropdownBtn');
        this.exportMenu = document.getElementById('exportMenu');
        this.copyListBtn = document.getElementById('copyListBtn');
        this.downloadCsvBtn = document.getElementById('downloadCsvBtn');

        // Dashboard Stats
        this.dashTotalItems = document.getElementById('dashTotalItems');
        this.dashCompletedPercent = document.getElementById('dashCompletedPercent');
        this.dashProgressBar = document.getElementById('dashProgressBar');
        this.dashEstTotal = document.getElementById('dashEstTotal');

        this.suggestionsContainer = document.getElementById('suggestionsContainer');
        this.seasonBadge = document.getElementById('seasonBadge');
        this.refreshSuggestionsBtn = document.getElementById('refreshSuggestionsBtn');

        this.searchInput = document.getElementById('searchInput');
        this.clearSearchBtn = document.getElementById('clearSearchBtn');
        this.activeFilterBanner = document.getElementById('activeFilterBanner');
        this.activeFilterText = document.getElementById('activeFilterText');
        this.resetFilterBtn = document.getElementById('resetFilterBtn');
```

```
        this.categoryFilterTabs = document.getElementById('categoryFilterTabs');

        this.shoppingListContainer = document.getElementById('shoppingListContainer');
        this.emptyListState = document.getElementById('emptyListState');
        this.itemCountBadge = document.getElementById('itemCountBadge');
        this.clearCompletedBtn = document.getElementById('clearCompletedBtn');
        this.clearAllBtn = document.getElementById('clearAllBtn');

        this.summaryTotalCount = document.getElementById('summaryTotalCount');
        this.summaryPendingCount = document.getElementById('summaryPendingCount');
        this.summaryTotalPrice = document.getElementById('summaryTotalPrice');

        // Modals
        this.addItemModal = document.getElementById('addItemModal');
        this.openAddItemModalBtn = document.getElementById('openAddItemModalBtn');
        this.closeAddItemModalBtn = document.getElementById('closeAddItemModalBtn');
        this.cancelAddItemModalBtn = document.getElementById('cancelAddItemModalBtn');
        this.manualAddForm = document.getElementById('manualAddForm');

        this.settingsModal = document.getElementById('settingsModal');
        this.openSettingsBtn = document.getElementById('openSettingsBtn');
        this.closeSettingsModalBtn = document.getElementById('closeSettingsModalBtn');
        this.saveSettingsBtn = document.getElementById('saveSettingsBtn');
        this.voiceFeedbackToggle = document.getElementById('voiceFeedbackToggle');
        this.soundFxToggle = document.getElementById('soundFxToggle');
        this.voiceSpeedRange = document.getElementById('voiceSpeedRange');
        this.speedValueText = document.getElementById('speedValueText');
        this.geminiApiKeyInput = document.getElementById('geminiApiKeyInput');
        this.toastContainer = document.getElementById('toastContainer');
    }

    /**
     * Initialize Voice Recognition engine
     */
    initVoice() {
        this.voiceManager = new VoiceManager(
            (result) => this.handleSpeechResult(result),
            (stateInfo) => this.handleVoiceStateChange(stateInfo)
        );

        const settings = StorageManager.getSettings();
        if (settings.theme === 'dark') {
            document.documentElement.classList.add('dark');
            this.themeIcon.setAttribute('data-lucide', 'sun');
        }
        if (settings.language) {
            this.languageSelect.value = settings.language;
            this.updateHudLangText(settings.language);
        }
    }

    /**
     * Attach UI Event Listeners
     */
    initEventListeners() {
        // Mic Button Click
        const triggerMic = () => {
            if (this.soundEnabled) SoundFX.playMicStart();
            this.voiceManager.startListening();
        };
        this.micBtn.addEventListener('click', triggerMic);
        this.hudMicBtn.addEventListener('click', triggerMic);

        // Language Select
        this.languageSelect.addEventListener('change', (e) => {
            const lang = e.target.value;
            this.voiceManager.setLanguage(lang);
            const settings = StorageManager.getSettings();
            settings.language = lang;
            StorageManager.saveSettings(settings);
            this.updateHudLangText(lang);
            this.showToast(`Language set to ${e.target.options[e.target.selectedIndex].text}`);
        });

        // Export Dropdown
        this.exportDropdownBtn.addEventListener('click', () => {
```

```
            this.exportMenu.classList.toggle('hidden');
        });

        document.addEventListener('click', (e) => {
            if (!this.exportDropdownBtn.contains(e.target) && !this.exportMenu.contains(e.targe
t)) {
                this.exportMenu.classList.add('hidden');
            }
        });

        this.copyListBtn.addEventListener('click', () => {
            this.exportAsText();
            this.exportMenu.classList.add('hidden');
        });

        this.downloadCsvBtn.addEventListener('click', () => {
            this.exportAsCSV();
            this.exportMenu.classList.add('hidden');
        });

        // Search Input & Reset
        this.searchInput.addEventListener('input', (e) => {
            this.searchQuery = e.target.value.toLowerCase().trim();
            this.clearSearchBtn.classList.toggle('hidden', !this.searchQuery);
            this.renderShoppingList();
        });

        this.clearSearchBtn.addEventListener('click', () => {
            this.searchInput.value = '';
            this.searchQuery = '';
            this.maxPriceFilter = null;
            this.clearSearchBtn.classList.add('hidden');
            this.activeFilterBanner.classList.add('hidden');
            this.renderShoppingList();
        });

        this.resetFilterBtn.addEventListener('click', () => {
            this.searchInput.value = '';
            this.searchQuery = '';
            this.maxPriceFilter = null;
            this.clearSearchBtn.classList.add('hidden');
            this.activeFilterBanner.classList.add('hidden');
            this.renderShoppingList();
        });

        // Category Filter Tabs
        this.categoryFilterTabs.addEventListener('click', (e) => {
            const btn = e.target.closest('.cat-tab');
            if (!btn) return;
            this.activeCategory = btn.getAttribute('data-category');
            this.updateCategoryTabStyles();
            this.renderShoppingList();
        });

        // Quick Hint Chips
        document.querySelectorAll('.hint-chip').forEach(chip => {
            chip.addEventListener('click', () => {
                const text = chip.textContent.replace(/"/g, '');
                this.processUtterance(text);
            });
        });

        // Recipe Bundle Buttons
        document.querySelectorAll('.recipe-bundle-btn').forEach(btn => {
            btn.addEventListener('click', () => {
                const recipeKey = btn.getAttribute('data-recipe');
                const items = RECIPE_BUNDLES[recipeKey];
                if (items) {
                    items.forEach(item => this.addItem(item, false));
                    if (this.soundEnabled) SoundFX.playItemAdd();
                    const recipeName = recipeKey.charAt(0).toUpperCase() + recipeKey.slice(1);
                    this.showToast(`Added ${recipeName} recipe ingredients to list!`);
                    this.voiceManager.speak(`Added ingredients for ${recipeName} to your list.`
);
                }
```

```
            });
        });
        // Clear buttons
        this.clearCompletedBtn.addEventListener('click', () => {
            this.items = this.items.filter(i => !i.completed);
            StorageManager.saveItems(this.items);
            this.render();
            if (this.soundEnabled) SoundFX.playDelete();
            this.showToast('Cleared completed items');
        });

        this.clearAllBtn.addEventListener('click', () => {
            if (confirm('Are you sure you want to clear all items from your shopping list?')) {
                this.items = [];
                StorageManager.saveItems(this.items);
                this.render();
                if (this.soundEnabled) SoundFX.playDelete();
                this.showToast('Shopping list cleared');
            }
        });

        // Theme Toggle
        this.themeToggleBtn.addEventListener('click', () => {
            const isDark = document.documentElement.classList.toggle('dark');
            const settings = StorageManager.getSettings();
            settings.theme = isDark ? 'dark' : 'light';
            StorageManager.saveSettings(settings);
            this.themeIcon.setAttribute('data-lucide', isDark ? 'sun' : 'moon');
            if (window.lucide) lucide.createIcons();
        });

        // Voice-Only HUD Mode
        this.toggleVoiceHudBtn.addEventListener('click', () => {
            this.voiceHudOverlay.classList.remove('hidden');
        });
        this.exitVoiceHudBtn.addEventListener('click', () => {
            this.voiceHudOverlay.classList.add('hidden');
        });

        // Add Item Modal
        this.openAddItemModalBtn.addEventListener('click', () => this.addItemModal.classList.re
move('hidden'));
        this.closeAddItemModalBtn.addEventListener('click', () => this.addItemModal.classList.a
dd('hidden'));
        this.cancelAddItemModalBtn.addEventListener('click', () => this.addItemModal.classList.
add('hidden'));
        this.manualAddForm.addEventListener('submit', (e) => {
            e.preventDefault();
            const name = document.getElementById('manualNameInput').value.trim();
            const quantity = parseFloat(document.getElementById('manualQtyInput').value) || 1;
            const unit = document.getElementById('manualUnitSelect').value;
            const category = document.getElementById('manualCategorySelect').value;
            const price = parseFloat(document.getElementById('manualPriceInput').value) || null
;
            const brand = document.getElementById('manualBrandInput').value.trim() || null;

            if (name) {
                this.addItem({ name, quantity, unit, category, price, brand });
                this.manualAddForm.reset();
                this.addItemModal.classList.add('hidden');
                if (this.soundEnabled) SoundFX.playItemAdd();
                this.showToast(`Added ${name} to list`);
                this.voiceManager.speak(`Added ${quantity} ${unit} of ${name} to your shopping
list.`);
            }
        });

        // Settings Modal
        this.openSettingsBtn.addEventListener('click', () => {
            const settings = StorageManager.getSettings();
            this.voiceFeedbackToggle.checked = settings.voiceFeedback;
            this.soundFxToggle.checked = this.soundEnabled;
            this.voiceSpeedRange.value = settings.voiceSpeed || 1.0;
            this.speedValueText.textContent = `${settings.voiceSpeed || 1.0}x`;
            this.geminiApiKeyInput.value = settings.geminiApiKey || '';
```

```
            this.settingsModal.classList.remove('hidden');
        });

        this.closeSettingsModalBtn.addEventListener('click', () => this.settingsModal.classList
.add('hidden'));

        this.voiceSpeedRange.addEventListener('input', (e) => {
            this.speedValueText.textContent = `${e.target.value}x`;
        });

        this.saveSettingsBtn.addEventListener('click', () => {
            const settings = StorageManager.getSettings();
            settings.voiceFeedback = this.voiceFeedbackToggle.checked;
            this.soundEnabled = this.soundFxToggle.checked;
            settings.voiceSpeed = parseFloat(this.voiceSpeedRange.value);
            settings.geminiApiKey = this.geminiApiKeyInput.value.trim();

            StorageManager.saveSettings(settings);
            this.settingsModal.classList.add('hidden');
            this.showToast('Settings saved successfully');
        });

        // Refresh Suggestions
        this.refreshSuggestionsBtn.addEventListener('click', () => {
            this.renderSuggestions();
            this.showToast('Refreshed smart recommendations');
        });
    }

    /**
     * Handle incoming raw speech recognition transcript
     */
    async handleSpeechResult(result) {
        if (result.interim) {
            this.transcriptText.textContent = `"${result.interim}"`;
            this.hudTranscriptBox.textContent = `"${result.interim}"`;
        }

        if (result.final) {
            const utterance = result.final;
            this.transcriptText.textContent = `"${utterance}"`;
            this.hudTranscriptBox.textContent = `"${utterance}"`;
            await this.processUtterance(utterance);
        }
    }

    /**
     * Process speech utterance through NLP Parser
     */
    async processUtterance(utteranceText) {
        this.setVoiceStatus('thinking', 'Processing command...');

        try {
            const action = await NLPParser.parse(utteranceText);

            if (!action || action.intent === 'UNKNOWN') {
                this.voiceManager.speak("I didn't quite catch that. Try saying add milk or find
 items under 5 dollars.");
                this.showToast('Command not recognized', 'error');
                return;
            }

            switch (action.intent) {
                case 'ADD_ITEM':
                    if (action.item && action.item.name) {
                        this.addItem(action.item);
                        if (this.soundEnabled) SoundFX.playItemAdd();
                        const msg = `Added ${action.item.quantity} ${action.item.name} to your
list.`;
                        this.voiceManager.speak(msg);
                        this.showToast(msg);
                    }
                    break;

                case 'ADD_RECIPE_BUNDLE':
                    if (action.items) {
                        action.items.forEach(i => this.addItem(i, false));
```

```
                        if (this.soundEnabled) SoundFX.playItemAdd();
                        const msg = `Added ${action.recipeName} ingredients to your shopping li
st.`;
                        this.voiceManager.speak(msg);
                        this.showToast(msg);
                    }
                    break;

                case 'REMOVE_ITEM':
                    if (action.item) {
                        const removed = this.removeItemByName(action.item);
                        if (removed) {
                            if (this.soundEnabled) SoundFX.playDelete();
                            const msg = `Removed ${action.item} from your shopping list.`;
                            this.voiceManager.speak(msg);
                            this.showToast(msg);
                        } else {
                            const msg = `Could not find ${action.item} on your list.`;
                            this.voiceManager.speak(msg);
                            this.showToast(msg, 'warning');
                        }
                    }
                    break;

                case 'TOGGLE_ITEM':
                    if (action.item) {
                        const toggled = this.toggleItemByName(action.item);
                        if (toggled) {
                            if (this.soundEnabled) SoundFX.playCheckPop();
                            const msg = `Marked ${action.item} as completed.`;
                            this.voiceManager.speak(msg);
                            this.showToast(msg);
                        }
                    }
                    break;

                case 'SEARCH':
                    this.searchQuery = (action.query || '').toLowerCase();
                    this.maxPriceFilter = action.maxPrice || null;
                    this.searchInput.value = this.searchQuery;
                    this.clearSearchBtn.classList.toggle('hidden', !this.searchQuery);

                    if (this.maxPriceFilter !== null) {
                        this.activeFilterText.textContent = `Filter: "${this.searchQuery || 'It
ems'}" under $${this.maxPriceFilter.toFixed(2)}`;
                        this.activeFilterBanner.classList.remove('hidden');
                        this.voiceManager.speak(`Filtered shopping list for items under ${this.
maxPriceFilter} dollars.`);
                    } else {
                        this.voiceManager.speak(`Searching for ${this.searchQuery}`);
                    }
                    this.renderShoppingList();
                    break;

                case 'SUGGESTIONS':
                    this.renderSuggestions();
                    this.voiceManager.speak(`Here are smart recommendations based on your histo
ry and seasonal favorites.`);
                    this.showToast('Generated smart recommendations');
                    break;
            }
        } catch (err) {
            console.error('Error executing voice command:', err);
            this.showToast('Error processing voice command', 'error');
        } finally {
            this.setVoiceStatus('idle', 'Tap mic or say command');
        }
    }

    /**
     * Add item to list & update storage
     */
    addItem(itemData, updateRender = true) {
        const existingIndex = this.items.findIndex(i => i.name.toLowerCase() === itemData.name.
toLowerCase());
```

```
        if (existingIndex !== -1) {
            this.items[existingIndex].quantity += itemData.quantity;
            if (itemData.price) this.items[existingIndex].price = itemData.price;
        } else {
            const newItem = {
                id: Date.now().toString() + Math.random().toString(36).substr(2, 4),
                name: itemData.name,
                quantity: itemData.quantity || 1,
                unit: itemData.unit || 'pcs',
                category: itemData.category || StorageManager.detectCategory(itemData.name),
                price: itemData.price || null,
                brand: itemData.brand || null,
                completed: false,
                createdAt: Date.now()
            };
            this.items.unshift(newItem);
        }

        StorageManager.saveItems(this.items);
        StorageManager.recordPurchase(itemData.name, itemData.category);
        if (updateRender) this.render();
    }

    /**
     * Remove item by name
     */
    removeItemByName(name) {
        const initialLength = this.items.length;
        this.items = this.items.filter(i => !i.name.toLowerCase().includes(name.toLowerCase()))
;
        if (this.items.length !== initialLength) {
            StorageManager.saveItems(this.items);
            this.render();
            return true;
        }
        return false;
    }

    /**
     * Toggle completion status by item name
     */
    toggleItemByName(name) {
        const item = this.items.find(i => i.name.toLowerCase().includes(name.toLowerCase()));
        if (item) {
            item.completed = !item.completed;
            StorageManager.saveItems(this.items);
            this.render();
            return true;
        }
        return false;
    }

    /**
     * Export Shopping List as Text
     */
    exportAsText() {
        if (this.items.length === 0) {
            this.showToast('Shopping list is empty', 'warning');
            return;
        }
        let text = `■ VoiceCart AI - Shopping List (${new Date().toLocaleDateString()})\n\n`;
        this.items.forEach((item, idx) => {
            const check = item.completed ? '[x]' : '[ ]';
            const priceStr = item.price ? ` ($${item.price.toFixed(2)})` : '';
            text += `${idx + 1}. ${check} ${item.name} - ${item.quantity} ${item.unit} [${item.
category}]${priceStr}\n`;
        });
        navigator.clipboard.writeText(text);
        this.showToast('Shopping list copied to clipboard!');
    }

    /**
     * Export Shopping List as CSV
     */
    exportAsCSV() {
```

```
        if (this.items.length === 0) {
            this.showToast('Shopping list is empty', 'warning');
            return;
        }
        let csv = 'Status,Name,Quantity,Unit,Category,Price,Brand\n';
        this.items.forEach(i => {
            const status = i.completed ? 'Completed' : 'Pending';
            csv += `"${status}","${i.name}",${i.quantity},"${i.unit}","${i.category}",${i.price
 || 0},"${i.brand || ''}"\n`;
        });

        const blob = new Blob([csv], { type: 'text/csv' });
        const url = URL.createObjectURL(blob);
        const a = document.createElement('a');
        a.href = url;
        a.download = `Shopping_List_${new Date().toISOString().slice(0, 10)}.csv`;
        a.click();
        URL.revokeObjectURL(url);
        this.showToast('Downloaded Shopping_List.csv');
    }

    handleVoiceStateChange({ state, message }) {
        this.setVoiceStatus(state, message);
    }

    setVoiceStatus(state, text) {
        this.voiceStatusText.textContent = text;

        if (state === 'listening') {
            this.micBtn.classList.add('mic-active');
            this.voiceStatusDot.className = 'w-2.5 h-2.5 rounded-full bg-rose-500 animate-ping'
;
            this.audioWaveform.classList.remove('hidden');
            this.audioWaveform.classList.add('flex');
            this.micSubtext.textContent = 'LISTENING...';
        } else if (state === 'speaking') {
            this.micBtn.classList.remove('mic-active');
            this.voiceStatusDot.className = 'w-2.5 h-2.5 rounded-full bg-emerald-500 animate-pu
lse';
            this.audioWaveform.classList.remove('hidden');
            this.audioWaveform.classList.add('flex');
            this.micSubtext.textContent = 'SPEAKING';
        } else if (state === 'thinking') {
            this.micBtn.classList.remove('mic-active');
            this.voiceStatusDot.className = 'w-2.5 h-2.5 rounded-full bg-amber-500 animate-spin
';
            this.audioWaveform.classList.add('hidden');
            this.audioWaveform.classList.remove('flex');
            this.micSubtext.textContent = 'PARSING...';
        } else {
            this.micBtn.classList.remove('mic-active');
            this.voiceStatusDot.className = 'w-2.5 h-2.5 rounded-full bg-indigo-500 animate-pul
se';
            this.audioWaveform.classList.add('hidden');
            this.audioWaveform.classList.remove('flex');
            this.micSubtext.textContent = 'TAP TO SPEAK';
        }
    }

    updateHudLangText(langCode) {
        const langMap = {
            'en-US': 'English (US)',
            'es-ES': 'Español (ES)',
            'fr-FR': 'Français (FR)',
            'de-DE': 'Deutsch (DE)',
            'hi-IN': '[redacted] (IN)',
            'zh-CN': '[redacted] (CN)'
        };
        this.hudLangText.textContent = langMap[langCode] || langCode;
    }

    updateCategoryTabStyles() {
        document.querySelectorAll('.cat-tab').forEach(btn => {
            const cat = btn.getAttribute('data-category');
            if (cat === this.activeCategory) {
                btn.className = 'cat-tab bg-indigo-600 text-white px-3.5 py-1.5 rounded-lg font
```

```
-medium whitespace-nowrap shadow-sm';
            } else {
                btn.className = 'cat-tab bg-slate-100 dark:bg-slate-800 text-slate-600 dark:tex
t-slate-300 px-3.5 py-1.5 rounded-lg font-medium hover:bg-slate-200 whitespace-nowrap';
            }
        });
    }

    renderShoppingList() {
        let filtered = [...this.items];

        if (this.activeCategory !== 'ALL') {
            filtered = filtered.filter(i => i.category === this.activeCategory);
        }

        if (this.searchQuery) {
            filtered = filtered.filter(i =>
                i.name.toLowerCase().includes(this.searchQuery) ||
                (i.brand && i.brand.toLowerCase().includes(this.searchQuery))
            );
        }

        if (this.maxPriceFilter !== null) {
            filtered = filtered.filter(i => i.price !== null && i.price <= this.maxPriceFilter)
;
        }

        this.itemCountBadge.textContent = `${filtered.length} items`;
        this.emptyListState.classList.toggle('hidden', filtered.length > 0);

        this.shoppingListContainer.innerHTML = filtered.map(item => this.createItemCardHTML(ite
m)).join('');

        this.shoppingListContainer.querySelectorAll('.item-checkbox').forEach(chk => {
            chk.addEventListener('change', (e) => {
                const id = e.target.getAttribute('data-id');
                const targetItem = this.items.find(i => i.id === id);
                if (targetItem) {
                    targetItem.completed = e.target.checked;
                    if (this.soundEnabled && targetItem.completed) SoundFX.playCheckPop();
                    StorageManager.saveItems(this.items);
                    this.render();
                }
            });
        });

        this.shoppingListContainer.querySelectorAll('.qty-minus-btn').forEach(btn => {
            btn.addEventListener('click', (e) => {
                const id = e.currentTarget.getAttribute('data-id');
                const targetItem = this.items.find(i => i.id === id);
                if (targetItem && targetItem.quantity > 1) {
                    targetItem.quantity -= 1;
                    StorageManager.saveItems(this.items);
                    this.render();
                }
            });
        });

        this.shoppingListContainer.querySelectorAll('.qty-plus-btn').forEach(btn => {
            btn.addEventListener('click', (e) => {
                const id = e.currentTarget.getAttribute('data-id');
                const targetItem = this.items.find(i => i.id === id);
                if (targetItem) {
                    targetItem.quantity += 1;
                    StorageManager.saveItems(this.items);
                    this.render();
                }
            });
        });

        this.shoppingListContainer.querySelectorAll('.delete-item-btn').forEach(btn => {
            btn.addEventListener('click', (e) => {
                const id = e.currentTarget.getAttribute('data-id');
                this.items = this.items.filter(i => i.id !== id);
                if (this.soundEnabled) SoundFX.playDelete();
```

```
                StorageManager.saveItems(this.items);
                this.render();
                this.showToast('Item deleted');
            });
        });

        // Summary Calculations & Dashboard Update
        const totalItemsCount = this.items.length;
        const completedCount = this.items.filter(i => i.completed).length;
        const pendingCount = totalItemsCount - completedCount;
        const totalEstPrice = this.items.reduce((sum, i) => sum + ((i.price || 0) * i.quantity)
, 0);
        const completedPercent = totalItemsCount > 0 ? Math.round((completedCount / totalItemsC
ount) * 100) : 0;

        this.dashTotalItems.textContent = `${totalItemsCount} Items`;
        this.dashCompletedPercent.textContent = `${completedPercent}%`;
        this.dashProgressBar.style.width = `${completedPercent}%`;
        this.dashEstTotal.textContent = `$${totalEstPrice.toFixed(2)}`;

        this.summaryTotalCount.textContent = totalItemsCount;
        this.summaryPendingCount.textContent = pendingCount;
        this.summaryTotalPrice.textContent = `$${totalEstPrice.toFixed(2)}`;

        if (window.lucide) lucide.createIcons();
    }

    createItemCardHTML(item) {
        const categoryBadgeClass = this.getBadgeClass(item.category);
        const isDoneClass = item.completed ? 'item-completed' : '';

        return `
            <div class="glass-card rounded-2xl p-4 flex items-center justify-between gap-3 ${is
DoneClass} hover:border-indigo-300 dark:hover:border-indigo-700 transition-all shadow-xs">
                <div class="flex items-center space-x-3.5 min-w-0">
                    <input type="checkbox" data-id="${item.id}" ${item.completed ? 'checked' :
''} class="item-checkbox w-5 h-5 text-indigo-600 rounded cursor-pointer accent-indigo-600">
                    <div class="min-w-0">
                        <div class="flex items-center space-x-2">
                            <span class="item-title font-semibold text-sm truncate">${item.name
}</span>
                            <span class="text-xs px-2.5 py-0.5 rounded-full font-medium ${categ
oryBadgeClass}">${item.category}</span>
                        </div>
                        <div class="text-xs text-slate-400 mt-0.5 flex items-center space-x-2">
                            ${item.brand ? `<span class="font-medium text-slate-500">Brand: ${i
tem.brand}</span><span>•</span>` : ''}
                            <span class="font-medium text-slate-500">Price: ${item.price ? `$${
item.price.toFixed(2)}` : 'N/A'}</span>
                        </div>
                    </div>
                </div>

                <div class="flex items-center space-x-3">
                    <div class="flex items-center space-x-1.5 bg-slate-100 dark:bg-slate-800 ro
unded-xl px-2.5 py-1 border border-slate-200/60 dark:border-slate-700/60">
                        <button data-id="${item.id}" class="qty-minus-btn text-slate-500 hover:
text-slate-900 dark:hover:text-white p-0.5">
                            <i data-lucide="minus" class="w-3.5 h-3.5"></i>
                        </button>
                        <span class="text-xs font-bold w-6 text-center">${item.quantity} ${item
.unit || ''}</span>
                        <button data-id="${item.id}" class="qty-plus-btn text-slate-500 hover:t
ext-slate-900 dark:hover:text-white p-0.5">
                            <i data-lucide="plus" class="w-3.5 h-3.5"></i>
                        </button>
                    </div>
                    <button data-id="${item.id}" class="delete-item-btn p-1.5 text-slate-400 ho
ver:text-rose-500 transition-colors">
                        <i data-lucide="trash-2" class="w-4 h-4"></i>
                    </button>
                </div>
            </div>
        `;
    }
```

```
    renderSuggestions() {
        const data = SuggestionsEngine.getAllSuggestions();
        this.seasonBadge.textContent = data.seasonName;

        let cardsHTML = '';

        data.history.forEach(item => {
            cardsHTML += `
                <div class="p-4 rounded-2xl bg-amber-500/10 border border-amber-500/20 text-xs
space-y-2 flex flex-col justify-between">
                    <div>
                        <div class="flex items-center justify-between mb-1">
                            <span class="font-bold text-amber-700 dark:text-amber-300">■ Low St
ock Warning</span>
                            <span class="px-2 py-0.5 rounded bg-amber-200 dark:bg-amber-900/60
text-[10px] text-amber-900 dark:text-amber-200 font-semibold">${item.category}</span>
                        </div>
                        <p class="font-semibold text-slate-800 dark:text-slate-100">${item.name
}</p>
                        <p class="text-slate-500 dark:text-slate-400 text-[11px] mt-0.5">${item
.reason}</p>
                    </div>
                    <button data-name="${item.name}" data-category="${item.category}" class="ad
d-suggestion-btn w-full mt-2 py-1.5 bg-amber-600 hover:bg-amber-700 text-white rounded-xl font-
medium flex items-center justify-center space-x-1 shadow-xs">
                        <i data-lucide="plus" class="w-3 h-3"></i>
                        <span>Add to List</span>
                    </button>
                </div>
            `;
        });

        data.seasonal.slice(0, 2).forEach(item => {
            cardsHTML += `
                <div class="p-4 rounded-2xl bg-emerald-500/10 border border-emerald-500/20 text
-xs space-y-2 flex flex-col justify-between">
                    <div>
                        <div class="flex items-center justify-between mb-1">
                            <span class="font-bold text-emerald-700 dark:text-emerald-300">■ In
 Season (${item.season})</span>
                            <span class="px-2 py-0.5 rounded bg-emerald-200 dark:bg-emerald-900
/60 text-[10px] text-emerald-900 dark:text-emerald-200 font-semibold">$${item.price.toFixed(2)}
</span>
                        </div>
                        <p class="font-semibold text-slate-800 dark:text-slate-100">${item.name
}</p>
                        <p class="text-slate-500 dark:text-slate-400 text-[11px] mt-0.5">${item
.reason}</p>
                    </div>
                    <button data-name="${item.name}" data-category="${item.category}" data-pric
e="${item.price}" class="add-suggestion-btn w-full mt-2 py-1.5 bg-emerald-600 hover:bg-emerald-
700 text-white rounded-xl font-medium flex items-center justify-center space-x-1 shadow-xs">
                        <i data-lucide="plus" class="w-3 h-3"></i>
                        <span>Add to List</span>
                    </button>
                </div>
            `;
        });

        data.substitutes.forEach(item => {
            cardsHTML += `
                <div class="p-4 rounded-2xl bg-purple-500/10 border border-purple-500/20 text-x
s space-y-2 flex flex-col justify-between">
                    <div>
                        <div class="flex items-center justify-between mb-1">
                            <span class="font-bold text-purple-700 dark:text-purple-300">■ Heal
thy Swap</span>
                            <span class="text-[10px] text-purple-500">For ${item.originalItem}<
/span>
                        </div>
                        <p class="font-semibold text-slate-800 dark:text-slate-100">${item.subs
titute}</p>
                        <p class="text-slate-500 dark:text-slate-400 text-[11px] mt-0.5">${item
.reason}</p>
                    </div>
```

```
                    <button data-name="${item.substitute}" class="add-suggestion-btn w-full mt-
2 py-1.5 bg-purple-600 hover:bg-purple-700 text-white rounded-xl font-medium flex items-center
justify-center space-x-1 shadow-xs">
                        <i data-lucide="plus" class="w-3 h-3"></i>
                        <span>Try Substitute</span>
                    </button>
                </div>
            `;
        });

        this.suggestionsContainer.innerHTML = cardsHTML;

        this.suggestionsContainer.querySelectorAll('.add-suggestion-btn').forEach(btn => {
            btn.addEventListener('click', (e) => {
                const name = e.currentTarget.getAttribute('data-name');
                const category = e.currentTarget.getAttribute('data-category');
                const price = parseFloat(e.currentTarget.getAttribute('data-price')) || null;

                this.addItem({ name, quantity: 1, category, price });
                if (this.soundEnabled) SoundFX.playItemAdd();
                this.showToast(`Added ${name} from suggestions`);
                this.voiceManager.speak(`Added ${name} to your list.`);
            });
        });

        if (window.lucide) lucide.createIcons();
    }

    getBadgeClass(category) {
        const map = {
            'Produce': 'badge-produce',
            'Dairy': 'badge-dairy',
            'Bakery': 'badge-bakery',
            'Meat & Seafood': 'badge-meat',
            'Pantry': 'badge-pantry',
            'Beverages': 'badge-beverages',
            'Snacks': 'badge-snacks',
            'Household': 'badge-household',
            'Personal Care': 'badge-personal'
        };
        return map[category] || 'badge-pantry';
    }

    showToast(message, type = 'info') {
        const toast = document.createElement('div');
        const bg = type === 'error' ? 'bg-rose-600' : type === 'warning' ? 'bg-amber-600' : 'bg
-slate-900 dark:bg-slate-100 text-white dark:text-slate-900';
        toast.className = `pointer-events-auto px-4 py-2.5 rounded-xl shadow-xl ${bg} text-xs f
ont-semibold flex items-center space-x-2 transition-all duration-300 transform translate-y-2 op
acity-0`;
        toast.innerHTML = `<span>${message}</span>`;

        this.toastContainer.appendChild(toast);
        requestAnimationFrame(() => {
            toast.classList.remove('translate-y-2', 'opacity-0');
        });

        setTimeout(() => {
            toast.classList.add('opacity-0', 'translate-y-2');
            setTimeout(() => toast.remove(), 300);
        }, 3000);
    }

    render() {
        this.renderShoppingList();
        this.renderSuggestions();
    }
}

document.addEventListener('DOMContentLoaded', () => {
    window.app = new App();
});
```

4. js/voice.js

Description: Web Speech Recognition & Text-to-Speech Synthesis Engine | File Path: js/voice.js

```
/**
 * Voice Manager - Handles Web Speech API (SpeechRecognition & SpeechSynthesis)
 * - Real-time continuous speech recognition
 * - Audio visualizer waveform state triggers
 * - Text-to-Speech audio feedback responses
 * - Multilingual locale switching
 */

import { StorageManager } from './storage.js';

export class VoiceManager {
    constructor(onResultCallback, onStateChangeCallback) {
        this.onResult = onResultCallback;
        this.onStateChange = onStateChangeCallback;
        this.recognition = null;
        this.isListening = false;
        this.isSpeaking = false;
        this.synth = window.speechSynthesis || null;

        this.initRecognition();
    }

    /**
     * Initialize browser Web Speech Recognition
     */
    initRecognition() {
        const SpeechRecognition = window.SpeechRecognition || window.webkitSpeechRecognition;
        if (!SpeechRecognition) {
            console.warn('SpeechRecognition API is not supported in this browser environment.')
;
            return;
        }

        this.recognition = new SpeechRecognition();
        this.recognition.continuous = false; // Capture discrete voice commands
        this.recognition.interimResults = true; // Real-time feedback as user speaks
        this.recognition.maxAlternatives = 1;

        const settings = StorageManager.getSettings();
        this.recognition.lang = settings.language || 'en-US';

        this.recognition.onstart = () => {
            this.isListening = true;
            this.notifyState('listening', 'Listening for command...');
        };
        this.recognition.onresult = (event) => {
            let interimTranscript = '';
            let finalTranscript = '';

            for (let i = event.resultIndex; i < event.results.length; ++i) {
                if (event.results[i].isFinal) {
                    finalTranscript += event.results[i][0].transcript;
                } else {
                    interimTranscript += event.results[i][0].transcript;
                }
            }

            if (this.onResult) {
                this.onResult({
                    final: finalTranscript,
                    interim: interimTranscript
                });
            }
        };

        this.recognition.onerror = (event) => {
            console.error('Speech recognition error:', event.error);
            this.isListening = false;
            this.notifyState('error', `Voice error: ${event.error}`);
        };
        this.recognition.onend = () => {
```

```
            this.isListening = false;
            this.notifyState('idle', 'Tap mic or say command');
        };
    }

    /**
     * Set active voice recognition language
     */
    setLanguage(langCode) {
        if (this.recognition) {
            this.recognition.lang = langCode;
        }
    }

    /**
     * Start speech listening session
     */
    startListening() {
        if (!this.recognition) {
            alert('Voice input is not supported in this browser. Please use Google Chrome, Micr
osoft Edge, or Safari.');
            return;
        }
        if (this.isListening) {
            this.stopListening();
            return;
        }
        try {
            // Stop any ongoing speech synthesis before listening
            if (this.synth && this.synth.speaking) {
                this.synth.cancel();
            }
            const settings = StorageManager.getSettings();
            this.recognition.lang = settings.language || 'en-US';
            this.recognition.start();
        } catch (e) {
            console.error('Error starting speech recognition:', e);
        }
    }

    /**
     * Stop speech listening
     */
    stopListening() {
        if (this.recognition && this.isListening) {
            this.recognition.stop();
            this.isListening = false;
            this.notifyState('idle', 'Stopped listening');
        }
    }

    /**
     * Speak audio confirmation back to the user (Text-to-Speech)
     */
    speak(text) {
        const settings = StorageManager.getSettings();
        if (!settings.voiceFeedback || !this.synth) return;

        // Cancel previous utterances
        this.synth.cancel();

        const utterance = new SpeechSynthesisUtterance(text);
        utterance.lang = settings.language || 'en-US';
        utterance.rate = settings.voiceSpeed || 1.0;

        utterance.onstart = () => {
            this.isSpeaking = true;
            this.notifyState('speaking', text);
        };

        utterance.onend = () => {
            this.isSpeaking = false;
            this.notifyState('idle', 'Ready');
        };
        utterance.onerror = (err) => {
```

```
            console.error('Speech synthesis error:', err);
            this.isSpeaking = false;
            this.notifyState('idle', 'Ready');
        };

        this.synth.speak(utterance);
    }

    /**
     * Notify UI controller of state changes (listening, speaking, processing, idle, error)
     */
    notifyState(state, message) {
        if (this.onStateChange) {
            this.onStateChange({ state, message });
        }
    }
}
```

5. js/nlp.js

Description: Natural Language Intent Parser & Recipe Bundles Engine | File Path: js/nlp.js

```
/**
 * NLP Engine - Natural Language Processing & Intent Parser
 * Supports:
 * - Smart client-side regex & fuzzy keyword pattern extraction
 * - Entity extraction: Item Name, Quantity, Unit, Category, Price Limit, Brand
 * - Recipe Bundle shortcuts (Pancakes, Guacamole, Salad, Tacos)
 * - Optional Google Gemini API integration for advanced context understanding
 * - Multilingual keyword translation map (EN, ES, FR, DE, HI, ZH)
 */

import { StorageManager } from './storage.js';

// Pre-configured recipe bundle ingredient packages
export const RECIPE_BUNDLES = {
    'pancake': [
        { name: 'All-Purpose Flour', quantity: 1, unit: 'bag', category: 'Pantry', price: 2.99
},
        { name: 'Organic Eggs', quantity: 12, unit: 'pcs', category: 'Meat & Seafood', price: 3
.99 },
        { name: 'Whole Milk', quantity: 1, unit: 'gallon', category: 'Dairy', price: 4.29 },
        { name: 'Unsalted Butter', quantity: 1, unit: 'pack', category: 'Dairy', price: 3.49 },
        { name: 'Maple Syrup', quantity: 1, unit: 'bottle', category: 'Pantry', price: 5.99 }
    ],
    'guacamole': [
        { name: 'Ripe Avocados', quantity: 4, unit: 'pcs', category: 'Produce', price: 4.99 },
        { name: 'Fresh Limes', quantity: 3, unit: 'pcs', category: 'Produce', price: 1.49 },
        { name: 'Red Onion', quantity: 1, unit: 'pcs', category: 'Produce', price: 0.99 },
        { name: 'Fresh Cilantro', quantity: 1, unit: 'bunch', category: 'Produce', price: 1.29
},
        { name: 'Tortilla Chips', quantity: 1, unit: 'bag', category: 'Snacks', price: 3.29 }
    ],
    'salad': [
        { name: 'Romaine Lettuce', quantity: 1, unit: 'head', category: 'Produce', price: 2.49
},
        { name: 'Cherry Tomatoes', quantity: 1, unit: 'pack', category: 'Produce', price: 2.99
},
        { name: 'Cucumber', quantity: 2, unit: 'pcs', category: 'Produce', price: 1.49 },
        { name: 'Extra Virgin Olive Oil', quantity: 1, unit: 'bottle', category: 'Pantry', pric
e: 7.99 }
    ],
    'taco': [
        { name: 'Ground Beef', quantity: 1, unit: 'lbs', category: 'Meat & Seafood', price: 5.9
9 },
        { name: 'Taco Shells', quantity: 1, unit: 'box', category: 'Pantry', price: 2.79 },
        { name: 'Shredded Cheese', quantity: 1, unit: 'pack', category: 'Dairy', price: 3.29 },
        { name: 'Salsa', quantity: 1, unit: 'jar', category: 'Pantry', price: 2.99 },
        { name: 'Sour Cream', quantity: 1, unit: 'tub', category: 'Dairy', price: 1.99 }
    ]
};

// Multilingual keyword dictionary for core verbs
```

```
const MULTILINGUAL_SYNONYMS = {
    add: ['add', 'need', 'buy', 'want', 'get', 'put', 'include', 'añadir', 'agregar', 'comprar'
, 'necesito', 'ajouter', 'acheter', 'hinzufügen', 'kaufen', 'jodo', 'chahiye', 'láo'],
    remove: ['remove', 'delete', 'take off', 'clear', 'drop', 'quitar', 'eliminar', 'borrar', '
supprimer', 'enlever', 'entfernen', 'löschen', 'hatao', 'ninkalo'],
    search: ['find', 'search', 'look for', 'show', 'where is', 'buscar', 'chercher', 'suchen',
'khojo', 'dhoondho'],
    suggest: ['suggest', 'recommend', 'what should i buy', 'suggestions', 'sugerir', 'sugerenci
as', 'suggérer', 'empfehlen', 'sujhav'],
    check: ['check', 'mark', 'done', 'complete', 'marcar', 'cocher', 'abhaken', 'karo']
};

export class NLPParser {
    /**
     * Parse raw voice text transcript into a structured action object
     */
    static async parse(transcript) {
        if (!transcript || typeof transcript !== 'string') {
            return { intent: 'UNKNOWN', originalText: transcript };
        }

        const text = transcript.trim().toLowerCase();

        // 0. Recipe Bundle Detection (e.g., "Add pancake ingredients", "Guacamole recipe")
        for (const [recipeKey, items] of Object.entries(RECIPE_BUNDLES)) {
            if (text.includes(recipeKey) && (text.includes('ingredient') || text.includes('reci
pe') || text.includes('bundle') || text.includes('kit') || text.includes('make') || text.includ
es('add'))) {
                return {
                    intent: 'ADD_RECIPE_BUNDLE',
                    recipeName: recipeKey.charAt(0).toUpperCase() + recipeKey.slice(1),
                    items: items,
                    originalText: text
                };
            }
        }

        // Check if Gemini API Key is configured in settings
        const settings = StorageManager.getSettings();

        if (settings.geminiApiKey && settings.geminiApiKey.trim() !== '') {
            try {
                const geminiResult = await this.parseWithGemini(text, settings.geminiApiKey);
                if (geminiResult && geminiResult.intent !== 'UNKNOWN') {
                    return geminiResult;
                }
            } catch (err) {
                console.warn('Gemini API parse failed, falling back to local NLP:', err);
            }
        }

        // Fallback to local high-performance regex & keyword parser
        return this.parseLocal(text);
    }

    /**
     * Local rule-based & Regex NLP parser
     */
    static parseLocal(text) {
        // 1. Search / Price Filter Intent
        if (this.hasKeyword(text, MULTILINGUAL_SYNONYMS.search) || text.includes('under $') ||
text.includes('under ') || text.includes('below ')) {
            const priceMatch = text.match(/(?:under|below|less than|\$)\s*(\d+(?:\.\d+)?)\s*(?:
dollars|\$)?/i);
            const priceCap = priceMatch ? parseFloat(priceMatch[1]) : null;

            let query = text
                .replace(/(?:find|search|look for|show me|under|below|less than|\$|\d+(?:\.\d+)
?|dollars)/gi, '')
                .replace(/\s+/g, ' ')
                .trim();

            return {
                intent: 'SEARCH',
                query: query || '',
                maxPrice: priceCap,
```

```
                originalText: text
            };
        }

        // 2. Suggestions Intent
        if (this.hasKeyword(text, MULTILINGUAL_SYNONYMS.suggest) || text.includes('running low'
) || text.includes('what to buy')) {
            return {
                intent: 'SUGGESTIONS',
                originalText: text
            };
        }

        // 3. Remove Intent
        if (this.hasKeyword(text, MULTILINGUAL_SYNONYMS.remove)) {
            const itemName = this.extractItemName(text, MULTILINGUAL_SYNONYMS.remove);
            return {
                intent: 'REMOVE_ITEM',
                item: itemName,
                originalText: text
            };
        }

        // 4. Check / Complete Intent
        if (this.hasKeyword(text, MULTILINGUAL_SYNONYMS.check)) {
            const itemName = this.extractItemName(text, MULTILINGUAL_SYNONYMS.check);
            return {
                intent: 'TOGGLE_ITEM',
                item: itemName,
                originalText: text
            };
        }

        // 5. Add / Modify Item Intent
        const quantity = this.extractQuantity(text);
        const unit = this.extractUnit(text);
        const price = this.extractPrice(text);
        const brand = this.extractBrand(text);
        let itemName = this.extractItemName(text, MULTILINGUAL_SYNONYMS.add);

        if (!itemName || itemName.length < 2) {
            itemName = text.replace(/^(?:add|buy|need|want|get|put)\s+/i, '').trim();
        }

        itemName = this.capitalizeWords(itemName);
        const category = StorageManager.detectCategory(itemName);

        return {
            intent: 'ADD_ITEM',
            item: {
                name: itemName,
                quantity: quantity || 1,
                unit: unit || 'pcs',
                category: category,
                price: price || null,
                brand: brand || null
            },
            originalText: text
        };
    }

    static hasKeyword(text, keywordList) {
        return keywordList.some(kw => text.includes(kw));
    }

    static extractQuantity(text) {
        const numberWords = { 'one': 1, 'two': 2, 'three': 3, 'four': 4, 'five': 5, 'six': 6, '
seven': 7, 'eight': 8, 'nine': 9, 'ten': 10 };
        const match = text.match(/\b(\d+(?:\.\d+)?)\b/);
        if (match) return parseFloat(match[1]);

        for (const [word, num] of Object.entries(numberWords)) {
            if (new RegExp(`\\b${word}\\b`, 'i').test(text)) {
                return num;
            }
```

```
        }
        return 1;
    }

    static extractUnit(text) {
        const units = ['bottle', 'bottles', 'gallon', 'gallons', 'pack', 'packs', 'box', 'boxes
', 'loaf', 'loaves', 'kg', 'lbs', 'lb', 'liter', 'liters', 'bag', 'bags', 'tube', 'tubes', 'can
', 'cans', 'carton', 'cartons', 'bunch', 'head', 'tub'];
        for (const u of units) {
            if (new RegExp(`\\b${u}\\b`, 'i').test(text)) {
                return u;
            }
        }
        return 'pcs';
    }

    static extractPrice(text) {
        const match = text.match(/(?:\$|for|at)\s*(\d+(?:\.\d+)?)\s*(?:dollars|\$)?/i);
        return match ? parseFloat(match[1]) : null;
    }

    static extractBrand(text) {
        const brands = ['organic', 'horizon', 'dave\'s', 'colgate', 'nestle', 'heinz', 'kellogg
\'s', 'tropicana', 'chobani', 'fairlife', 'silk'];
        for (const b of brands) {
            if (new RegExp(`\\b${b}\\b`, 'i').test(text)) {
                return this.capitalizeWords(b);
            }
        }
        return null;
    }

    static extractItemName(text, verbList) {
        let cleaned = text;

        verbList.forEach(verb => {
            cleaned = cleaned.replace(new RegExp(`\\b${verb}\\b`, 'gi'), '');
        });

        cleaned = cleaned
            .replace(/\b(?:i|my|the|a|an|some|to|on|list|from|please|for|me|shopping|under|doll
ars|\$|\d+(?:\.\d+)?|pcs|bottles?|gallons?|packs?|boxes?|loaf|loaves|kg|lbs|liters?|bags?|tubes
?|cans?|cartons?)\b/gi, '')
            .replace(/\s+/g, ' ')
            .trim();

        return cleaned;
    }

    static capitalizeWords(str) {
        if (!str) return '';
        return str.replace(/\w\S*/g, txt => txt.charAt(0).toUpperCase() + txt.substr(1).toLower
Case());
    }

    static async parseWithGemini(text, apiKey) {
        const endpoint = `https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-fl
ash:generateContent?key=${apiKey}`;
        const prompt = `You are a voice command shopping list assistant NLP parser.
Analyze utterance: "${text}".
Return ONLY valid JSON matching this schema:
{
  "intent": "ADD_ITEM" | "REMOVE_ITEM" | "SEARCH" | "SUGGESTIONS" | "TOGGLE_ITEM",
  "item": {
     "name": "Product Name",
     "quantity": 1,
     "unit": "pcs",
     "category": "Produce/Dairy/Bakery/Pantry/Beverages/Snacks/Personal Care/Household/Meat & S
eafood",
     "price": 0.00 or null,
     "brand": "Brand Name" or null
  },
  "query": "search query string or null",
  "maxPrice": 0.00 or null
}`;
```

```
        const response = await fetch(endpoint, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ contents: [{ parts: [{ text: prompt }] }] })
        });

        const data = await response.json();
        const responseText = data.candidates?.[0]?.content?.parts?.[0]?.text;
        if (responseText) {
            const jsonMatch = responseText.match(/\{[\s\S]*\}/);
            if (jsonMatch) return JSON.parse(jsonMatch[0]);
        }
        return null;
    }
}
```

6. js/suggestions.js

Description: Smart Recommendations, History & Product Substitutes Engine | File Path: js/suggestions.js

```
/**
 * Suggestions Engine - Generates smart recommendations based on:
 * 1. Shopping history (frequent/low stock items)
 * 2. Seasonal availability (Spring/Summer/Autumn/Winter)
 * 3. Smart product substitutes (e.g. Almond Milk for Whole Milk)
 */

import { StorageManager } from './storage.js';

// Database of smart substitutes
const SUBSTITUTES_DB = {
    'whole milk': [
        { name: 'Almond Milk', reason: 'Plant-based alternative with lower calories', type: 'Di
etary' },
        { name: 'Oat Milk', reason: 'Creamy texture, great for coffee', type: 'Dietary' },
        { name: 'Lactose-Free Milk', reason: 'Easier to digest option', type: 'Health' }
    ],
    'milk': [
        { name: 'Almond Milk', reason: 'Popular plant-based alternative', type: 'Dietary' },
        { name: 'Oat Milk', reason: 'Rich & creamy alternative', type: 'Dietary' }
    ],
    'white bread': [
        { name: 'Whole Wheat Bread', reason: 'Higher fiber and nutritional value', type: 'Healt
h' },
        { name: 'Sourdough Bread', reason: 'Easier on digestion and natural gut health', type:
'Health' }
    ],
    'bread': [
        { name: 'Whole Grain Bread', reason: 'More fiber & essential nutrients', type: 'Health'
 }
    ],
    'sugar': [
        { name: 'Honey', reason: 'Natural sweetener with antioxidants', type: 'Health' },
        { name: 'Stevia', reason: 'Zero calorie natural sweetener', type: 'Dietary' }
    ],
    'soda': [
        { name: 'Sparkling Water with Lemon', reason: 'Zero sugar refreshing drink', type: 'Hea
lth' },
        { name: 'Kombucha', reason: 'Probiotic-rich gut health drink', type: 'Health' }
    ],
    'potato chips': [
        { name: 'Baked Kale Chips', reason: 'Lower calorie crunchy snack', type: 'Health' },
        { name: 'Air-popped Popcorn', reason: 'Whole grain low-fat snack', type: 'Dietary' }
    ],
    'butter': [
        { name: 'Olive Oil', reason: 'Heart-healthy unsaturated fats', type: 'Health' },
        { name: 'Avocado Oil', reason: 'High smoke point & healthy fats', type: 'Health' }
    ]
};

// Seasonal Database
const SEASONAL_ITEMS = {
    Spring: [
```

```
        { name: 'Fresh Strawberries', category: 'Produce', price: 3.49, reason: 'Peak Spring ha
rvest & on sale' },
        { name: 'Organic Asparagus', category: 'Produce', price: 2.99, reason: 'Fresh Spring cr
op' },
        { name: 'Green Peas', category: 'Produce', price: 1.99, reason: 'In season now' }
    ],
    Summer: [
        { name: 'Fresh Watermelon', category: 'Produce', price: 4.99, reason: 'Summer favorite
& hydrating' },
        { name: 'Sweet Corn', category: 'Produce', price: 0.50, reason: 'Fresh local Summer har
vest' },
        { name: 'Peaches', category: 'Produce', price: 2.49, reason: 'Juicy summer pick' },
        { name: 'Ice Cream', category: 'Snacks', price: 3.99, reason: 'Summer beat-the-heat dea
l' }
    ],
    Autumn: [
        { name: 'Honeycrisp Apples', category: 'Produce', price: 2.99, reason: 'Autumn orchard
fresh' },
        { name: 'Pumpkin Spice Latte Mix', category: 'Beverages', price: 4.99, reason: 'Fall se
asonal favorite' },
        { name: 'Butternut Squash', category: 'Produce', price: 1.89, reason: 'Fresh Fall harve
st' }
    ],
    Winter: [
        { name: 'Navel Oranges', category: 'Produce', price: 3.29, reason: 'Winter Vitamin C bo
ost' },
        { name: 'Hot Cocoa Mix', category: 'Beverages', price: 2.79, reason: 'Warm winter treat
' },
        { name: 'Clementines', category: 'Produce', price: 4.49, reason: 'Peak winter citrus se
ason' }
    ]
};

export class SuggestionsEngine {
    /**
     * Determine current season
     */
    static getCurrentSeason() {
        const month = new Date().getMonth(); // 0-11
        if (month >= 2 && month <= 4) return 'Spring';
        if (month >= 5 && month <= 7) return 'Summer';
        if (month >= 8 && month <= 10) return 'Autumn';
        return 'Winter';
    }

    /**
     * Get low stock / frequent item recommendations based on user history
     */
    static getHistoryRecommendations() {
        const history = StorageManager.getHistory();
        const currentItems = StorageManager.getItems().map(i => i.name.toLowerCase());
        const recommendations = [];
        const now = Date.now();

        for (const item of history) {
            // If item is not currently in shopping list
            if (!currentItems.includes(item.name.toLowerCase())) {
                const daysSinceBought = (now - item.lastBought) / (1000 * 60 * 60 * 24);
                // If frequent item (>2 times) and bought more than 5 days ago
                if (item.count >= 2 && daysSinceBought > 5) {
                    recommendations.push({
                        name: item.name,
                        category: item.category,
                        reason: `You usually buy this every week. Running low? (${Math.round(da
ysSinceBought)} days ago)`,
                        type: 'Low Stock'
                    });
                }
            }
        }
        return recommendations;
    }

    /**
     * Get seasonal recommendations
```

```
     */
    static getSeasonalRecommendations() {
        const season = this.getCurrentSeason();
        const currentItems = StorageManager.getItems().map(i => i.name.toLowerCase());
        const list = SEASONAL_ITEMS[season] || SEASONAL_ITEMS.Summer;

        return list.filter(item => !currentItems.includes(item.name.toLowerCase())).map(item =>
({
            ...item,
            season: season,
            type: 'Seasonal'
        }));
    }

    /**
     * Find smart substitute options for a given product or item list
     */
    static getSubstitutesForItem(itemName) {
        const lower = itemName.toLowerCase().trim();
        for (const [key, list] of Object.entries(SUBSTITUTES_DB)) {
            if (lower.includes(key)) {
                return list;
            }
        }
        return [];
    }

    /**
     * Generate all active smart suggestions combined
     */
    static getAllSuggestions() {
        const historyRecs = this.getHistoryRecommendations();
        const seasonalRecs = this.getSeasonalRecommendations();

        // Find substitutes for current items in the list
        const currentItems = StorageManager.getItems();
        const substituteRecs = [];

        for (const item of currentItems) {
            const subs = this.getSubstitutesForItem(item.name);
            if (subs.length > 0) {
                substituteRecs.push({
                    originalItem: item.name,
                    substitute: subs[0].name,
                    reason: subs[0].reason,
                    type: 'Substitute'
                });
            }
        }

        return {
            history: historyRecs,
            seasonal: seasonalRecs,
            substitutes: substituteRecs,
            seasonName: this.getCurrentSeason()
        };
    }
}
```

7. js/storage.js

Description: LocalStorage Persistence Layer & Pre-loaded Product DB | File Path: js/storage.js

```
/**
 * Storage Manager - Handles persistence using LocalStorage
 * Stores shopping list items, purchase history, preferences, and default seed data.
 */

const STORAGE_KEYS = {
    ITEMS: 'vcs_shopping_list_items',
    HISTORY: 'vcs_purchase_history',
    SETTINGS: 'vcs_user_settings'
};

// Initial seed categories and product database
export const CATEGORY_MAP = {
    // Produce
    'apple': 'Produce', 'apples': 'Produce', 'banana': 'Produce', 'bananas': 'Produce',
    'orange': 'Produce', 'oranges': 'Produce', 'berry': 'Produce', 'berries': 'Produce',
    'strawberry': 'Produce', 'strawberries': 'Produce', 'lemon': 'Produce', 'lemons': 'Produce'
,
    'tomato': 'Produce', 'tomatoes': 'Produce', 'potato': 'Produce', 'potatoes': 'Produce',
    'onion': 'Produce', 'onions': 'Produce', 'garlic': 'Produce', 'lettuce': 'Produce',
    'spinach': 'Produce', 'carrot': 'Produce', 'carrots': 'Produce', 'avocado': 'Produce',
    'avocados': 'Produce', 'cucumber': 'Produce', 'cucumbers': 'Produce', 'mango': 'Produce',
    'watermelon': 'Produce', 'grape': 'Produce', 'grapes': 'Produce',

    // Dairy & Alternatives
    'milk': 'Dairy', 'cheese': 'Dairy', 'butter': 'Dairy', 'yogurt': 'Dairy',
    'cream': 'Dairy', 'almond milk': 'Dairy', 'oat milk': 'Dairy', 'soy milk': 'Dairy',
    'cottage cheese': 'Dairy', 'curd': 'Dairy', 'ghee': 'Dairy',

    // Bakery
    'bread': 'Bakery', 'bagel': 'Bakery', 'bagels': 'Bakery', 'croissant': 'Bakery',
    'muffin': 'Bakery', 'muffins': 'Bakery', 'bun': 'Bakery', 'buns': 'Bakery',
    'tortilla': 'Bakery', 'tortillas': 'Bakery', 'pita': 'Bakery',

    // Meat & Seafood
    'chicken': 'Meat & Seafood', 'beef': 'Meat & Seafood', 'pork': 'Meat & Seafood',
    'fish': 'Meat & Seafood', 'salmon': 'Meat & Seafood', 'shrimp': 'Meat & Seafood',
    'turkey': 'Meat & Seafood', 'bacon': 'Meat & Seafood', 'steak': 'Meat & Seafood',
    'eggs': 'Meat & Seafood', 'egg': 'Meat & Seafood',

    // Pantry & Grains
    'rice': 'Pantry', 'pasta': 'Pantry', 'flour': 'Pantry', 'sugar': 'Pantry',
    'salt': 'Pantry', 'pepper': 'Pantry', 'oil': 'Pantry', 'olive oil': 'Pantry',
    'cereal': 'Pantry', 'oats': 'Pantry', 'oatmeal': 'Pantry', 'beans': 'Pantry',
    'soup': 'Pantry', 'sauce': 'Pantry', 'ketchup': 'Pantry', 'mustard': 'Pantry',
    'honey': 'Pantry', 'peanut butter': 'Pantry', 'jam': 'Pantry',

    // Beverages
    'water': 'Beverages', 'juice': 'Beverages', 'coffee': 'Beverages', 'tea': 'Beverages',
    'soda': 'Beverages', 'coke': 'Beverages', 'wine': 'Beverages', 'beer': 'Beverages',
    'sparkling water': 'Beverages', 'lemonade': 'Beverages',

    // Snacks & Sweets
    'chips': 'Snacks', 'chocolate': 'Snacks', 'cookies': 'Snacks', 'nuts': 'Snacks',
    'almonds': 'Snacks', 'popcorn': 'Snacks', 'candy': 'Snacks', 'crackers': 'Snacks',
    'pretzels': 'Snacks',

    // Household & Personal Care
    'toothpaste': 'Personal Care', 'soap': 'Personal Care', 'shampoo': 'Personal Care',
    'paper towel': 'Household', 'paper towels': 'Household', 'toilet paper': 'Household',
    'detergent': 'Household', 'dish soap': 'Household', 'sponge': 'Household',
    'trash bags': 'Household', 'tissues': 'Household'
};

const DEFAULT_ITEMS = [
    { id: '1', name: 'Organic Milk', quantity: 1, unit: 'gallon', category: 'Dairy', price: 4.4
9, brand: 'Horizon', completed: false, createdAt: Date.now() - 3600000 },
    { id: '2', name: 'Fresh Apples', quantity: 6, unit: 'pcs', category: 'Produce', price: 3.99
, brand: 'Honeycrisp', completed: false, createdAt: Date.now() - 7200000 },
    { id: '3', name: 'Whole Wheat Bread', quantity: 1, unit: 'loaf', category: 'Bakery', price:
 2.99, brand: 'Dave\'s Killer', completed: true, createdAt: Date.now() - 10800000 },
```

```
    { id: '4', name: 'Colgate Toothpaste', quantity: 2, unit: 'tubes', category: 'Personal Care
', price: 4.29, brand: 'Colgate', completed: false, createdAt: Date.now() - 14400000 }
];

const DEFAULT_HISTORY = [
    { name: 'Organic Milk', category: 'Dairy', count: 5, lastBought: Date.now() - 864000000 },
// 10 days ago
    { name: 'Whole Wheat Bread', category: 'Bakery', count: 4, lastBought: Date.now() - 6048000
00 }, // 7 days ago
    { name: 'Fresh Apples', category: 'Produce', count: 3, lastBought: Date.now() - 432000000 }
, // 5 days ago
    { name: 'Eggs', category: 'Meat & Seafood', count: 6, lastBought: Date.now() - 1209600000 }
, // 14 days ago (Low stock warning!)
    { name: 'Bananas', category: 'Produce', count: 4, lastBought: Date.now() - 950400000 }
];

export class StorageManager {
    static getItems() {
        try {
            const data = localStorage.getItem(STORAGE_KEYS.ITEMS);
            if (!data) {
                this.saveItems(DEFAULT_ITEMS);
                return DEFAULT_ITEMS;
            }
            return JSON.parse(data);
        } catch (e) {
            console.error('Error loading items from localStorage:', e);
            return DEFAULT_ITEMS;
        }
    }

    static saveItems(items) {
        try {
            localStorage.setItem(STORAGE_KEYS.ITEMS, JSON.stringify(items));
        } catch (e) {
            console.error('Error saving items to localStorage:', e);
        }
    }

    static getHistory() {
        try {
            const data = localStorage.getItem(STORAGE_KEYS.HISTORY);
            if (!data) {
                this.saveHistory(DEFAULT_HISTORY);
                return DEFAULT_HISTORY;
            }
            return JSON.parse(data);
        } catch (e) {
            console.error('Error loading history:', e);
            return DEFAULT_HISTORY;
        }
    }

    static saveHistory(history) {
        try {
            localStorage.setItem(STORAGE_KEYS.HISTORY, JSON.stringify(history));
        } catch (e) {
            console.error('Error saving history:', e);
        }
    }

    static recordPurchase(itemName, category) {
        const history = this.getHistory();
        const existing = history.find(h => h.name.toLowerCase() === itemName.toLowerCase());
        if (existing) {
            existing.count += 1;
            existing.lastBought = Date.now();
        } else {
            history.push({
                name: itemName,
                category: category || this.detectCategory(itemName),
                count: 1,
                lastBought: Date.now()
            });
        }
```

```
        this.saveHistory(history);
    }

    static getSettings() {
        try {
            const data = localStorage.getItem(STORAGE_KEYS.SETTINGS);
            return data ? JSON.parse(data) : { language: 'en-US', voiceFeedback: true, voiceSpe
ed: 1, theme: 'light', geminiApiKey: '' };
        } catch (e) {
            return { language: 'en-US', voiceFeedback: true, voiceSpeed: 1, theme: 'light', gem
iniApiKey: '' };
        }
    }

    static saveSettings(settings) {
        try {
            localStorage.setItem(STORAGE_KEYS.SETTINGS, JSON.stringify(settings));
        } catch (e) {
            console.error('Error saving settings:', e);
        }
    }

    static detectCategory(itemName) {
        const lower = itemName.toLowerCase();
        for (const [key, category] of Object.entries(CATEGORY_MAP)) {
            if (lower.includes(key)) {
                return category;
            }
        }
        return 'Pantry'; // Default fallback
    }
}
```

8. js/sound.js

Description: Web Audio API SoundFX Synthesizer | File Path: js/sound.js

```
/**
 * Sound FX Manager - Web Audio API Sound Synthesizer
 * Generates clean, modern UI audio feedback with ZERO external audio files.
 */

export class SoundFX {
    static ctx = null;

    static getContext() {
        if (!this.ctx) {
            const AudioContext = window.AudioContext || window.webkitAudioContext;
            if (AudioContext) {
                this.ctx = new AudioContext();
            }
        }
        if (this.ctx && this.ctx.state === 'suspended') {
            this.ctx.resume();
        }
        return this.ctx;
    }

    /**
     * Mic Start Listening Tone (Soft dual ascending frequencies)
     */
    static playMicStart() {
        try {
            const ctx = this.getContext();
            if (!ctx) return;

            const now = ctx.currentTime;
            const osc = ctx.createOscillator();
            const gain = ctx.createGain();

            osc.type = 'sine';
            osc.frequency.setValueAtTime(440, now); // A4
            osc.frequency.exponentialRampToValueAtTime(880, now + 0.15); // A5

            gain.gain.setValueAtTime(0.08, now);
            gain.gain.exponentialRampToValueAtTime(0.001, now + 0.2);

            osc.connect(gain);
            gain.connect(ctx.destination);

            osc.start(now);
            osc.stop(now + 0.2);

        } catch (e) {
            console.warn('Audio play failed:', e);
        }
    }

    /**
     * Item Add Success Chime (Major triad arpeggio)
     */
    static playItemAdd() {
        try {
            const ctx = this.getContext();
            if (!ctx) return;

            const notes = [523.25, 659.25, 783.99]; // C5, E5, G5
            notes.forEach((freq, idx) => {
                const now = ctx.currentTime + (idx * 0.06);
                const osc = ctx.createOscillator();
                const gain = ctx.createGain();

                osc.type = 'triangle';
                osc.frequency.setValueAtTime(freq, now);

                gain.gain.setValueAtTime(0.06, now);
                gain.gain.exponentialRampToValueAtTime(0.001, now + 0.15);

                osc.connect(gain);
```

```
                gain.connect(ctx.destination);

                osc.start(now);
                osc.stop(now + 0.15);
            });
        } catch (e) {
            console.warn('Audio play failed:', e);
        }
    }

    /**
     * Item Complete Check Pop
     */
    static playCheckPop() {
        try {
            const ctx = this.getContext();
            if (!ctx) return;

            const now = ctx.currentTime;
            const osc = ctx.createOscillator();
            const gain = ctx.createGain();

            osc.type = 'sine';
            osc.frequency.setValueAtTime(600, now);
            osc.frequency.exponentialRampToValueAtTime(300, now + 0.08);

            gain.gain.setValueAtTime(0.08, now);
            gain.gain.exponentialRampToValueAtTime(0.001, now + 0.09);

            osc.connect(gain);
            gain.connect(ctx.destination);

            osc.start(now);
            osc.stop(now + 0.09);
        } catch (e) {
            console.warn('Audio play failed:', e);
        }
    }

    /**
     * Delete Item Soft Swoosh
     */
    static playDelete() {
        try {
            const ctx = this.getContext();
            if (!ctx) return;

            const now = ctx.currentTime;
            const osc = ctx.createOscillator();
            const gain = ctx.createGain();

            osc.type = 'sawtooth';
            osc.frequency.setValueAtTime(300, now);
            osc.frequency.exponentialRampToValueAtTime(100, now + 0.1);

            gain.gain.setValueAtTime(0.05, now);
            gain.gain.exponentialRampToValueAtTime(0.001, now + 0.11);

            osc.connect(gain);
            gain.connect(ctx.destination);

            osc.start(now);
            osc.stop(now + 0.11);
        } catch (e) {
            console.warn('Audio play failed:', e);
        }
    }
}
```

9. README.md

Description: Project Overview & 200-Word Approach Write-Up | File Path: README.md

````
# VoiceCart AI - Voice Command Shopping Assistant

> A modern, voice-activated shopping list manager with natural language intent understanding, s
mart recommendations, seasonal insights, product substitutes, and multilingual voice recognitio
n.

---

## ■ Approach Write-Up (Technical Summary - 142 Words)

VoiceCart AI is built as a zero-dependency, high-performance web application designed for fast,
 accessible, and intuitive voice-based shopping list management. It leverages the browser's nat
ive Web Speech API for real-time speech recognition and text-to-speech feedback, providing zero
 latency and no server overhead. Commands are parsed using a smart hybrid Natural Language Proc
essing (NLP) pipeline combining a client-side regex/entity extractor with an optional Google Ge
mini AI API integration. The system automatically extracts product names, quantities, units, br
ands, categories, and price thresholds, handling varied natural phrasing across 6 languages. To
 enhance user utility, VoiceCart features a Smart Suggestions Engine providing low-stock alerts
 based on shopping history, seasonal product highlights, and dietary substitute recommendations
. The minimalist responsive UI includes a dedicated Hands-Free Voice HUD, dark mode, live wavef
orm visualizer, and LocalStorage persistence for an optimal voice-first shopping experience.

---

## ■ Features Overview

### 1. Voice Input & Speech Synthesis
- **Voice Command Recognition**: Real-time microphone listening via Web Speech API (`SpeechReco
gnition`).
- **Text-to-Speech Feedback**: Speaks audio confirmations after executing actions (`speechSynth
esis`).
- **Multilingual Support**: Supports English, Spanish, French, German, Hindi, and Chinese voice
 commands.
- **Audio Waveform & HUD**: Interactive mic visualizer and full-screen **Hands-Free Voice HUD M
ode**.

### 2. Natural Language Processing (NLP)
- Parses natural utterances without requiring strict command syntax:
  - *"Add 2 bottles of organic milk for $4"*
  - *"Put 6 bananas on my list"*
  - *"Remove bread"*
  - *"Find items under $5"*
  - *"What should I buy?"*
- Auto-extracts: **Product Name**, **Quantity**, **Measurement Unit**, **Category**, **Price Li
mit**, and **Brand**.
- **Gemini AI Integration**: Optional setting to supply a Gemini API key for advanced conversat
ional intent parsing.

### 3. Smart Suggestions Engine
- **Product Recommendations**: Low-stock alerts based on purchase history and frequency.
- **Seasonal Recommendations**: Contextual seasonal items (Spring, Summer, Autumn, Winter).
- **Dietary & Product Substitutes**: Healthier alternatives (e.g. Almond Milk for Milk, Whole W
heat for White Bread).

### 4. Shopping List Management
- Automatic item categorization (*Produce, Dairy, Bakery, Meat, Pantry, Beverages, Snacks, Pers
onal Care, Household*).
- Quantity modification (+/-) and status toggling (completion state).
- Total estimated price calculation & item counters.
- Search bar with live keyword and price filter cap (*"items under $5"*).

---

## ■■ Architecture & Project Structure

```
voice-commanding/
■■■ index.html        # Main Application Interface & Modals
■■■ styles.css        # Visualizer Animations, Theme Styles & HUD Overlay
■■■ README.md         # Documentation & Approach Write-Up
■■■ .gitignore        # Submission Cleanup Rules
````
````

```
■■■ js/
    ■■■ app.js        # Main Controller & UI Event Dispatcher
    ■■■ voice.js      # SpeechRecognition & SpeechSynthesis Wrapper
    ■■■ nlp.js        # Hybrid Intent & Entity Extraction Engine
    ■■■ suggestions.js# Smart History, Seasonal & Substitutes Engine
    ■■■ storage.js    # LocalStorage Persistence & Category DB
```

■ How to Run Locally

Because VoiceCart AI is built with modern ES Modules and native web standards, it requires **ze
ro installation** or build steps!

1. **Option A: Python HTTP Server (Recommended)**
   ```bash
   python -m http.server 8000
   ```
   Open `http://localhost:8000` in your web browser (**Google Chrome**, **Microsoft Edge**, or
Safari recommended for speech recognition).

2. **Option B: VS Code Live Server**
   Right-click `index.html` and select **"Open with Live Server"**.

■ Checklist Compliance & Best Practices
- ■ **No extra dependencies or `node_modules`** (native browser APIs & minimal Tailwind CDN).
- ■ **Clean code structure** separated into modular ES classes (`VoiceManager`, `NLPParser`, `S
uggestionsEngine`, `StorageManager`).
- ■ **Clean git state** with `.gitignore` for build files and `.env`.
- ■ **Main branch ready** for public GitHub repository submission.

10. .gitignore

Description: Git Ignore Rules | File Path: .gitignore

```
# Git Ignore File
.DS_Store
Thumbs.db
*.log
.env
.vscode/
.idea/
node_modules/
dist/
build/
```